

Web Applications A.Y. 2025-2026
Homework 1 – Server-side Design and Development

Master Degree in Computer Engineering
Master Degree in Cybersecurity
Master Degree in ICT for Internet and Multimedia

Deadline: 17 April, 2026

Group Acronym	SWAD	
Last Name	First Name	Badge Number
Baldan	Fabio	2203580
Garberino	Alvise	2196387
Merja	Klaudio	2197815
Padoan	Giancarlo	2188345
Sanavia	Thomas	2197484

1 Objectives

The primary objective of this project is to design and develop a user-friendly web application that enables a bar owner to sell products online.

The system should provide an intuitive interface for customers to browse items, place orders and complete the transactions. It aims to streamline order processing for the bar owner. The application should be accessible across different devices and its goal is to enhance customer convenience while increasing the bar's sales and operational efficiency.

2 Main Functionalities

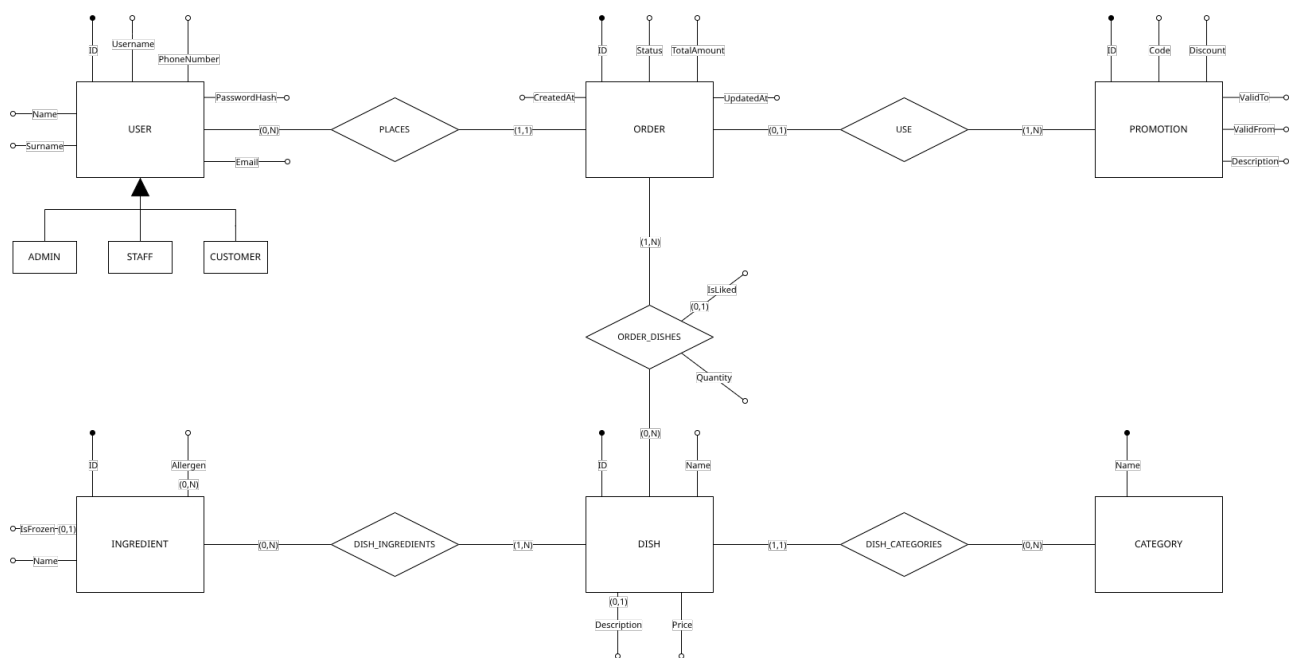
This web application will primarily allow registered customers-users to browse the bar's available products, including drinks and food and place orders online with ease.

Different features and actions may be available depending on the type of user account (e.g. regular customers or admin/staff users).

Some examples of this features are the possibility to buy dishes from the dashboard, to update the product catalogue, to manage incoming, completed orders and products availability, all with the goal of improving service efficiency. Every feature will be described better in the following chapters with the use of mock-up images, diagrams and REST-API calls examples.

3 Data Logic Layer

3.1 Entity-Relationship Schema



The enumerated domains used by the schema:

- **USER_ROLE** is used by `users.role`, it is mandatory and it restricts the allowed values to **STAFF**, **CUSTOMER**, and **ADMIN**.
- **ORDER_STATUS** is used by `orders.status`, it is also mandatory, and it restricts the allowed values to **PENDING**, **COMPLETED**, and **CANCELLED**.

- **ALLERGEN** is the domain used inside the array attribute `ingredients.allergen`, and it enumerates the allergen categories supported by the application.

We have 8 tables in the schema, including the associative entities introduced by the relational design:

- **User (users)**: each user is defined by `id`, which is of type `SERIAL`, mandatory and is the primary key of the entity. Then we have `username` of type `VARCHAR(20)`, which is mandatory and unique and stores the public username chosen by the user. We also have `email` of type `VARCHAR(255)`, mandatory and unique, which stores the email address associated with the account. The attributes `name` and `surname` are both of type `VARCHAR(255)`, both mandatory and they store respectively the user's first name and last name. The attribute `phone_number` is of type `VARCHAR(25)`, it is mandatory and unique and it stores the phone number associated with the account. The attribute `role` is of type `USER_ROLE`, it is mandatory and it represents the logical role of the user inside the application. The attribute `password_hash` is of type `VARCHAR(255)`, it is mandatory and it stores the hashed password used for authentication. In the current schema, customers, staff members, and administrators are not modelled as separate entities but are distinguished through the value of `role`.
- **Promotion (promotions)**: each promotion is defined by `id`, which is of type `SERIAL`, mandatory and is the primary key of the entity. Then we have `code`, of type `VARCHAR(20)`, which is mandatory, unique and stores the promotion code that can be applied to an order. The attribute `discount` is of type `FLOAT`, it is mandatory and it stores the percentage discount that must be applied when the promotion is used. The attribute `description` is of type `TEXT`, it is mandatory and it stores a textual description of the promotion. Finally, `valid_from` and `valid_to` are both of type `TIMESTAMP`, both mandatory and they store the beginning and the end of the validity interval of the promotion respectively. Moreover, the relational schema enforces the constraints `valid_to > valid_from` and `discount > 0 AND discount <= 100`. In this way the ending time of the promotion must always happen later than its starting time and the discount percentage must always be within the valid range.
- **Order (orders)**: each order is defined by `id`, which is of type `SERIAL`, mandatory and is the primary key of the entity. Then we have `user_id` of type `INTEGER`, which is mandatory and is a foreign key toward `users(id)` with `ON DELETE CASCADE`. It identifies the user who placed the order. The attribute `promotion_id` is of type `INTEGER`, it is optional and it is a foreign key toward `promotions(id)` with `ON DELETE SET NULL`. It identifies the promotion applied to the order (if any). The attribute `total_amount` is of type `DECIMAL(10, 2)`, it is mandatory and it stores the total monetary amount of the order. In the REST API this field is serialized as `total_price` in JSON. In the REST API this field is serialized as `total_price` in JSON. The attribute `status` is of type `ORDER_STATUS`, it is mandatory, it defaults to `PENDING` and it represents the current state of the order. In conclusion, `created_at` and `updated_at` are both of type `TIMESTAMP WITH TIME ZONE`, both mandatory and both initialized with the current timestamp. The attribute `updated_at` is also maintained by the `orders_set_updated_at` trigger before each update. Finally, the pair `(user_id, promotion_id)` is constrained to be unique when a promotion is referenced.
- **Category (categories)**: each category is defined only by `name`, which is of type `VARCHAR(255)`, mandatory, unique and is the primary key of the entity. In the current relational schema, therefore, the category name itself identifies the record and no numeric identifier is used.
- **Dish (dishes)**: each dish is defined by `id`, which is of type `SERIAL`, mandatory and is the primary key of the entity. Then we have `name`, of type `VARCHAR(255)`, which is mandatory and stores the commercial name of the dish. The attribute `description` is of type `TEXT`, it is optional and it stores an extended textual description of the dish. The attribute `category_name` is of type `VARCHAR(255)`, it is mandatory and it is a foreign key toward `categories(name)`. It identifies the category to which the dish belongs. The attribute `price` is of type `DECIMAL(10, 2)`, it is mandatory and it stores the current price of the dish.

- **OrderDish (order_dishes)**: this associative entity represents the many-to-many relationship between orders and dishes. Each row is defined by `id`, which is of type `SERIAL`, mandatory and is the primary key of the entity. Then we have `order_id`, of type `INTEGER`, which is mandatory and is a foreign key toward `orders(id)` with `ON DELETE CASCADE`: it identifies the order to which the row belongs. We also have `dish_id`, of type `INTEGER`, which is mandatory and is a foreign key toward `dishes(id)`: it identifies the ordered dish. The attribute `quantity` is of type `INTEGER`, it is mandatory and it stores how many units of the dish are included in the order. Finally, `is_liked` is of type `BOOLEAN`, it is optional and it stores feedback on the ordered dish. `TRUE` means that the dish was liked, `FALSE` means that it was disliked, and `NULL` means that no feedback has been provided yet. The pair `(order_id, dish_id)` is constrained to be unique, so the same dish can appear at most once in the same order.
- **Ingredient (ingredients)**: each ingredient is defined by `id`, which is of type `SERIAL`, mandatory and is the primary key of the entity. Then we have `name`, of type `VARCHAR(255)`, which is mandatory and stores the ingredient name. The attribute `allergen` is of type `ALLERGEN[]`, it is optional and it stores the list of allergens associated with the ingredient (if any). The attribute may be null if no allergen information is stored for that ingredient. In the REST API this field is serialized as `allergens` (plural) in JSON. The attribute `is_frozen` is of type `BOOLEAN`, it is optional and it indicates whether the ingredient is frozen.
- **DishIngredient (dish_ingredients)**: this associative entity represents the many-to-many relationship between dishes and ingredients. Each row is defined by `id`, which is of type `SERIAL`, mandatory and is the primary key of the entity. Then we have `dish_id` of type `INTEGER`, which is mandatory and is a foreign key toward `dishes(id)` with `ON DELETE CASCADE`: it identifies the dish. Finally, `ingredient_id` is of type `INTEGER`, it is mandatory and it is a foreign key toward `ingredients(id)` with `ON DELETE CASCADE`: it identifies the ingredient used in the dish. The pair `(dish_id, ingredient_id)` is constrained to be unique, so the same ingredient cannot be associated with the same dish more than once.

The relationships represented by the current schema can be summarized as follows. A user can place many orders while each order is associated with exactly one user through `orders.user_id`. A promotion can be used by many orders while each order can reference at most one promotion through `orders.promotion_id`. The relational schema also enforces that the same non-null pair `(user_id, promotion_id)` cannot be repeated. Each dish belongs to exactly one category through `dishes.category_name` while each category can be associated with many dishes. Orders and dishes are connected by a many-to-many relationship implemented through `order_dishes` and this associative entity also stores relationship-specific information such as `quantity` and `is_liked`. Dishes and ingredients are connected by a many-to-many relationship implemented through `dish_ingredients`.

3.2 Other Information

Below there is a list of important points that could be useful to know:

- Most entity identifiers are of type `SERIAL` (auto-incrementing integer), which guarantees uniqueness and offers efficient joins on numeric keys. The main exception is `categories` whose primary key is the attribute `name` of type `VARCHAR(255)`.
- Passwords are never stored in plain text. The `password_hash` column in the `users` table contains a bcrypt hash generated at registration time using the `BCrypt` library. At authentication time the submitted password is checked against the stored hash using `BCrypt.checkpw`.
- The `price` column in the `dishes` table stores monetary values as `DECIMAL(10,2)`. In the Java business logic layer, the `Dish` resource represents the price as a `Double`, while `CreateDishRR` validates that the price is present and strictly positive before delegating the insertion to the DAO layer.

- The `is_liked` field in `order_dishes` is nullable: it is intended to be populated only after an order reaches the `COMPLETED` status, when the customer can rate a dish. As of the current implementation, no database trigger enforces this rule: it is the responsibility of the application layer to avoid setting this field on non-completed orders.
- The `allergen` column in `ingredients` is a PostgreSQL array of the custom `ALLERGEN` enum type which lists all 14 allergens defined by EU food regulation. This allows a single ingredient to carry multiple allergens without introducing an additional join table.
- The application connects to PostgreSQL through a container-managed `DataSource` looked up via JNDI (`java:comp/env/jdbc/taptable`), configured in Tomcat's `context.xml`. A dedicated `taptable_manager` database role with limited privileges (no superuser, no create database, no create role, no inherit) is used for all application queries.
- The JWT signing secret is configured as a `context-param` named `jwt.secret` in `web.xml`. The current value is a placeholder (`change-me-in-production-use-a-long-random-string`) that must be replaced with a strong random secret before any production deployment.

4 Presentation Logic Layer

The web application is partially accessible to unauthenticated users, guests can visit the home page and the menu catalogue but any action that requires an account such as placing an order and accessing the dashboard redirects to the login page. The application exposes four JSP views, used for testing and developing purposes: ingredients, login, register and index (representation of the dashboard).

4.1 Home Page

The home page (figure 16) is the entry point for all visitors. It presents a brief introduction to the service and provides navigation links to the login and registration pages. No authentication is required and if there are some meals available that day they are displayed in an overview-table.

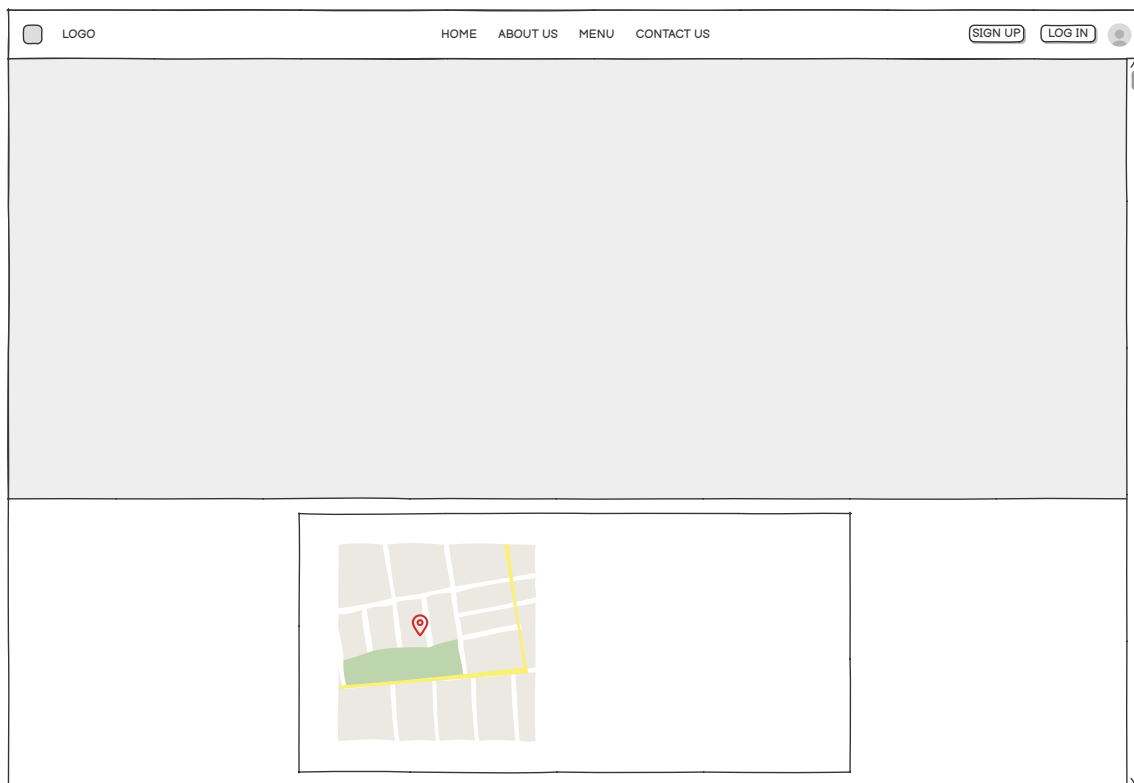


Figure 1: Home page's mockup

4.2 Login Page

The login page (figure 2) allows existing users to authenticate. The form collects the user's **email** and **password**. On submission, the credentials are sent via `POST /rest/user/login`. A successful login results in an `HttpOnly` cookie (`auth_token`) containing a signed JWT and consequently the user is redirected to the dashboard. If authentication fails, an error message is displayed inline.

LOGO

HOME ABOUT US MENU CONTACT US

LOG IN



[Don't have an account? Register](#)

Figure 2: Log in's page mockup

4.3 Registration Page

The registration page (figure 3) allows new customers to create an account. The form collects the following fields: **username**, **email**, **name**, **surname**, **phone number**, and **password**. All fields are mandatory so the user can't go on with the registration until all of them are filled. The password must satisfy a security policy: 8-16 characters, at least one uppercase letter, one lowercase letter, one digit and one special character. This policy is enforced server-side by the `Validator` utility class. `Validator` utility class validates also the email field: it allows the first part of the e-mail address to contain one or more characters (letters, digits and symbols + _ . -), followed by an @, a valid domain name, a dot and a top-level domain of at least two letters. On successful registration the user is redirected to the login page.

LOGO HOME ABOUT US MENU CONTACT US

SIGN UP

FIRST NAME

LAST NAME

USERNAME

EMAIL

REPEAT EMAIL

PHONE NUMBER

REPEAT PHONE NUMBER

PASSWORD

REPEAT PASSWORD

☐ I do accept the [Terms & Condition](#)

Already have an account? [Login](#)

Figure 3: Example of compilation of registration's page mockup

4.4 Dashboard

The dashboard (figures 4 - 5 - 6 - 7 - 8) is the main interface for authenticated users. The `AuthFilter` servlet filter is designed to protect all routes under `/dashboard/*` by validating the `auth_token` `HttpOnly` cookie on every request. If the token is missing or expired, it redirects to `/login` otherwise it sets the `user_id` request attribute. The dashboard will expose the menu catalogue, order management and staff/admin back-office functionality depending on the role of the authenticated user (`CUSTOMER`, `STAFF`, or `ADMIN`).

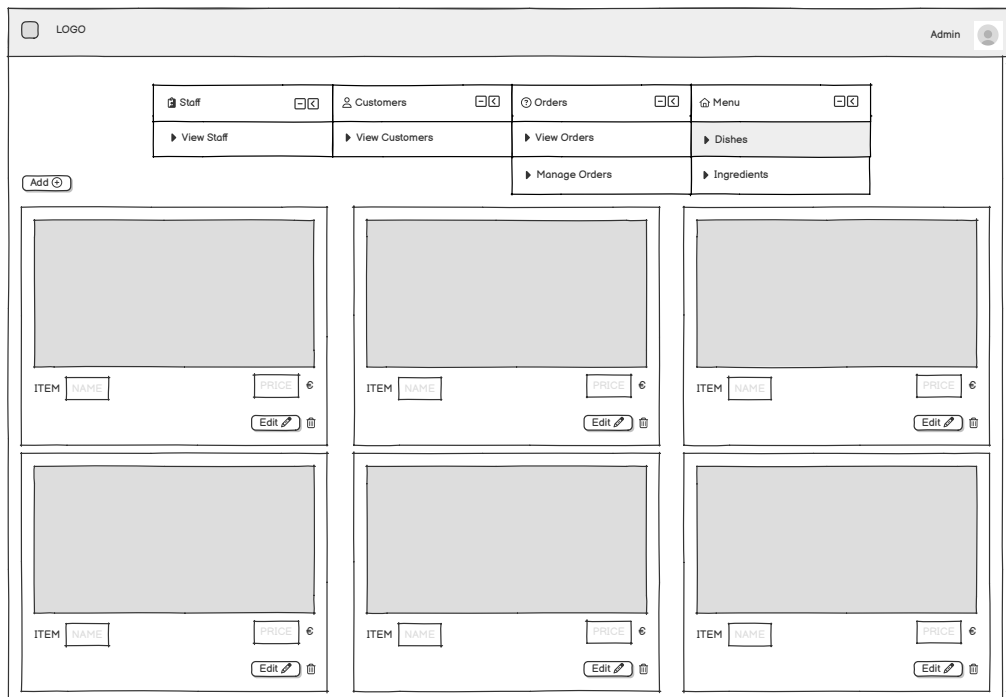


Figure 4: Management admin-side dishes dashboard mockup page.

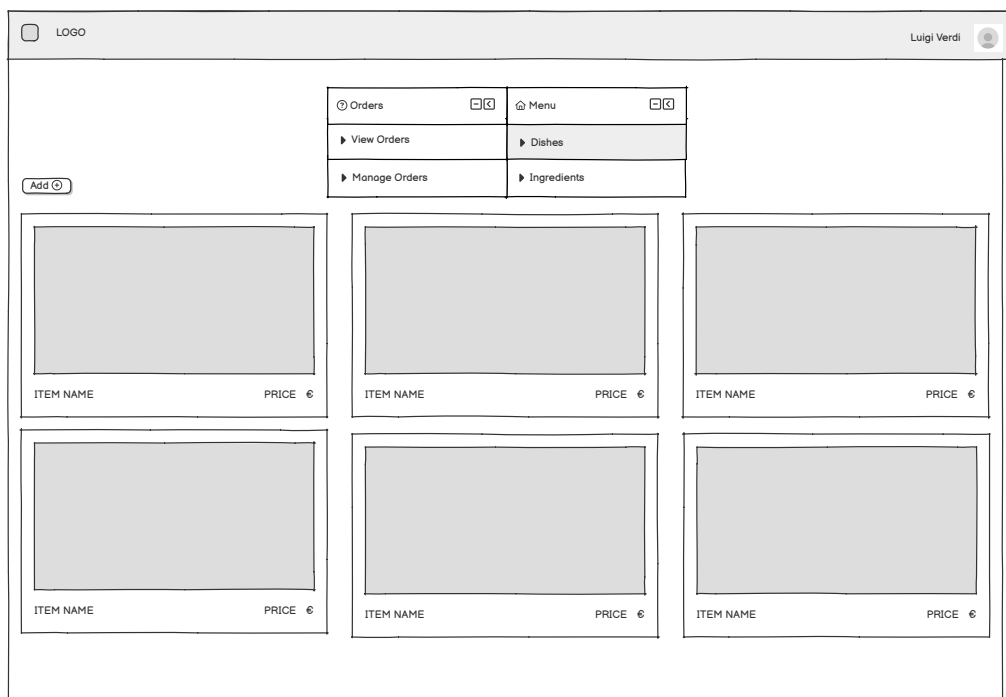


Figure 5: Management staff-side dishes dashboard mockup page.

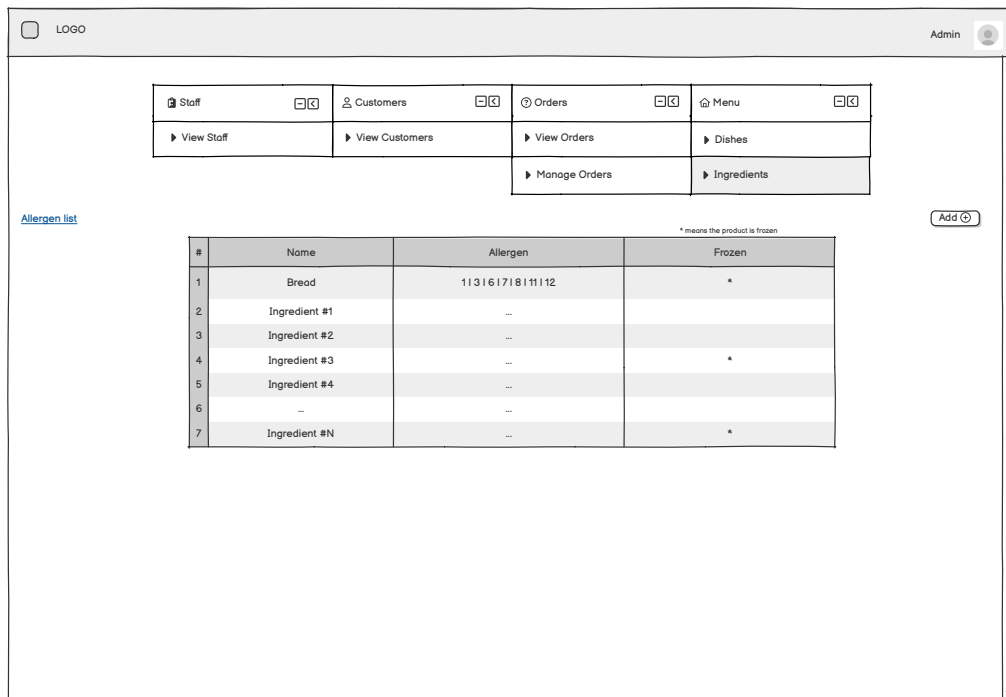


Figure 6: Dish's ingredients admin-side mockup page.

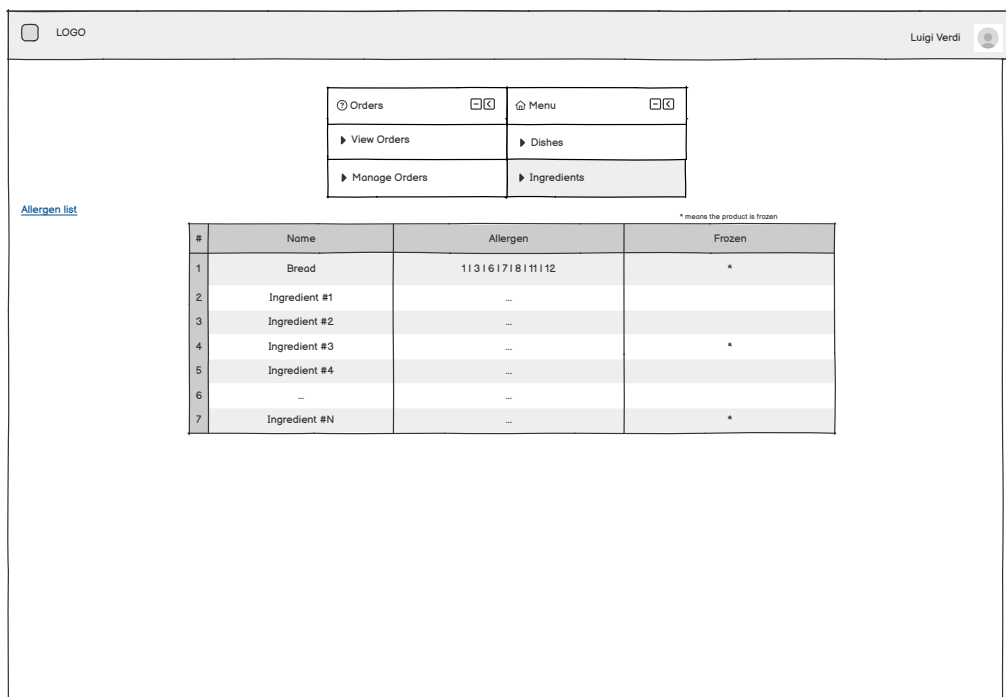


Figure 7: Dish's ingredients staff-side mockup page.



Figure 8: Menu dashboard's mockup page.

4.5 Other mockups

The following mockups complete the presentation layer by showing the secondary pages used by customers, staff members and administrators. They focus on cart handling, order tracking, account management and the back-office tools needed to supervise users and orders.

Figure 9: Profile's mockup page. The profile settings page allows users to consult and update their personal account information. It keeps account-related actions separated from the ordering flow while remaining accessible from the authenticated area.

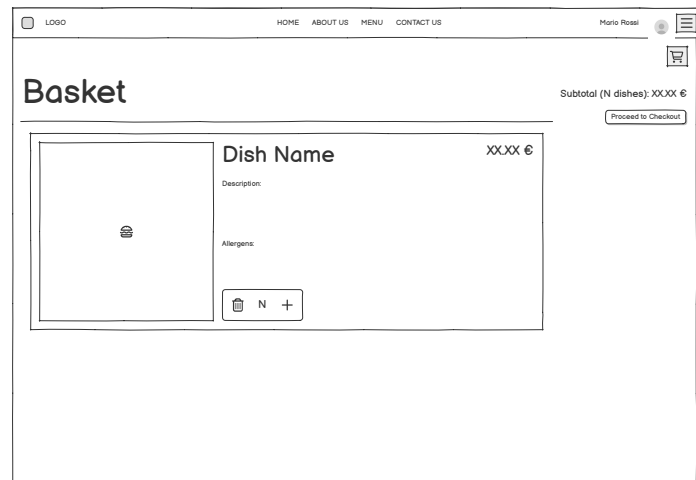


Figure 10: Shopping cart mockup page. The shopping cart page lets customers review the dishes selected from the menu before submitting an order. It provides a compact summary of the chosen items, quantities, and total amount, allowing the user to adjust the order before confirmation.

Figure 11: Orders' view custom-side mockup page.

The mockup shows a web page with a navigation bar and user profile. The main heading is 'Orders'. Below it is a table with the following data:

#	Date	State	Promotion Code	Note	Total	Reviewed
1	GGJHJYU/AAA A.HHJHJH	Completed	XXXX-XXXX	-	XXXX €	☒
2	GGJHJYU/AAA A.HHJHJH	Completed	None	-	XXXX €	☒
3	GGJHJYU/AAA A.HHJHJH	Completed	None	-	XXXX €	☒
4	GGJHJYU/AAA A.HHJHJH	Pending	XXXX-XXXX	-	XXXX €	☐

On the right side, there are links for 'Sign out', 'Profile', 'Settings', 'Orders', 'View Orders', and 'Review Orders'.

(a) Customer's orders mockup page. The customer's orders page allows authenticated customers to monitor their own orders. It shows the relevant order information and helps users understand the current state of each request.

The mockup shows a web page with a navigation bar and user profile. The main heading is 'Review Orders'. Below it is a table with the following data:

Date	Ordered dishes	Total	Review
GGJHJYU/AAA.HHJHJH	- Dish #1 - Dish #2 - Dish #3	XXXX €	☒ 4/5
GGJHJYU/AAA.HHJHJH	- Dish #1 - Dish #2	XXXX €	Not Available
GGJHJYU/AAA.HHJHJH	- Dish #1 - Dish #2 - Dish #3 - Dish #4	XXXX €	☒ 4/5
GGJHJYU/AAA.HHJHJH	- Dish #1	XXXX €	☐ 4/5

On the right side, there are links for 'Sign out', 'Profile', 'Settings', 'Orders', 'View Orders', and 'Review Orders'.

(b) Orders' review mockup page. The order review page gives customers the opportunity to express their opinion on their orders.

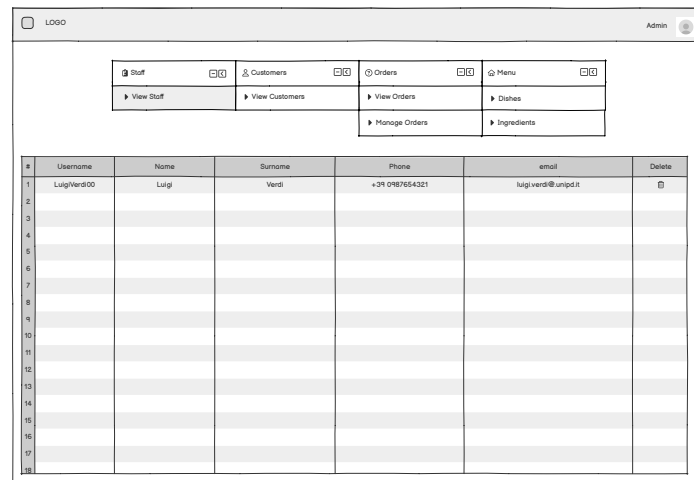


Figure 12: Staff management's mockup page. The staff management page is reserved for administrators and supports the supervision of staff accounts. It represents the interface used to manage internal users.

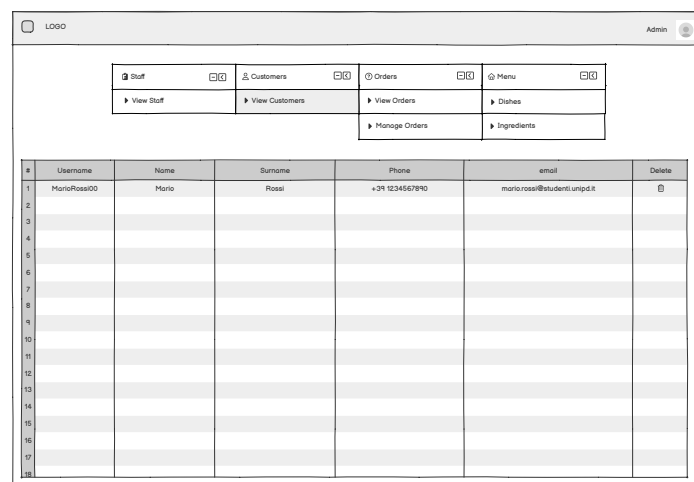
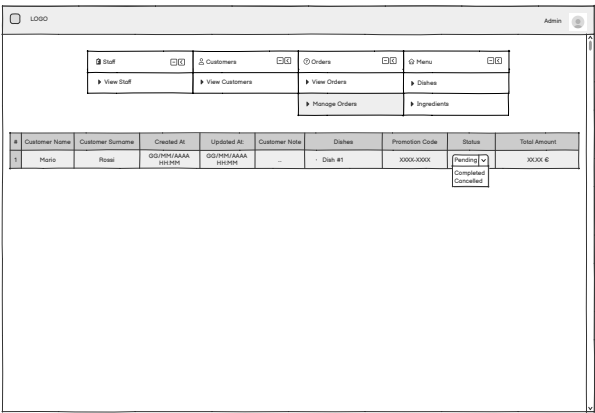
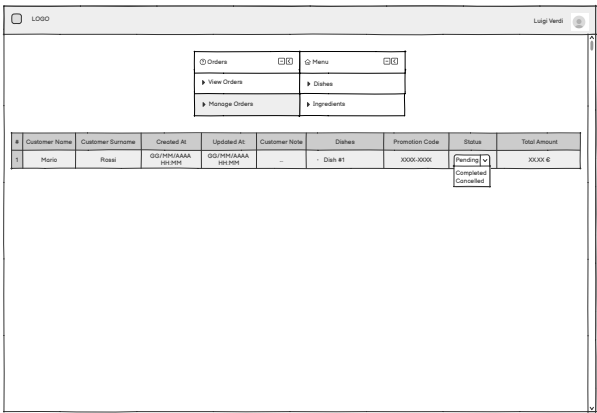


Figure 13: Customers' management mockup page. The customers' management page is intended for administrative users. It provides an overview of registered customers and supports the supervision of customer data from the back-office area.

Figure 14: Management status' mockup page.

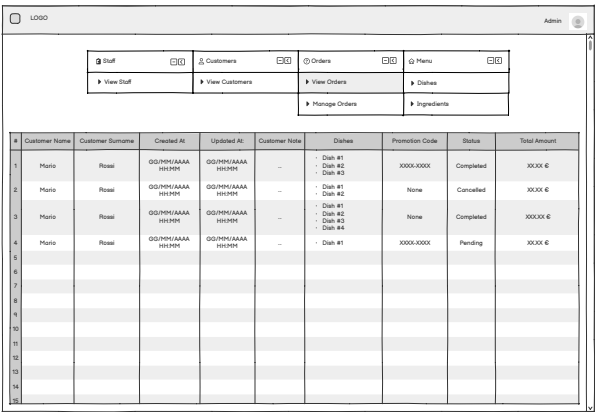


(a) Management admin-side status' mockup page. The admin-side orders management page gives administrators a global view of order statuses and exposes the controls required to coordinate the order workflow.

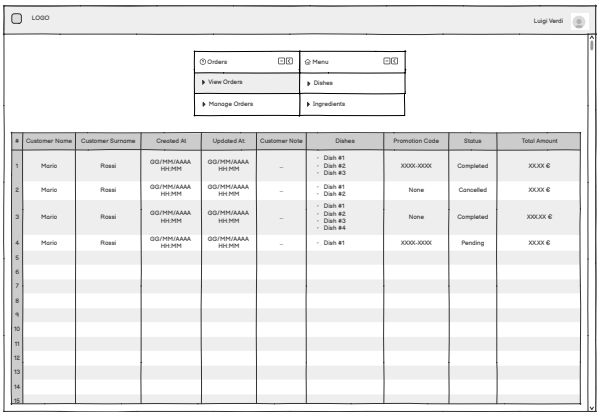


(b) Management staff-side status' mockup page. The staff-side orders management page focuses on the operational tasks assigned to staff members.

Figure 15: Orders' view admin/staff-side mockup page.



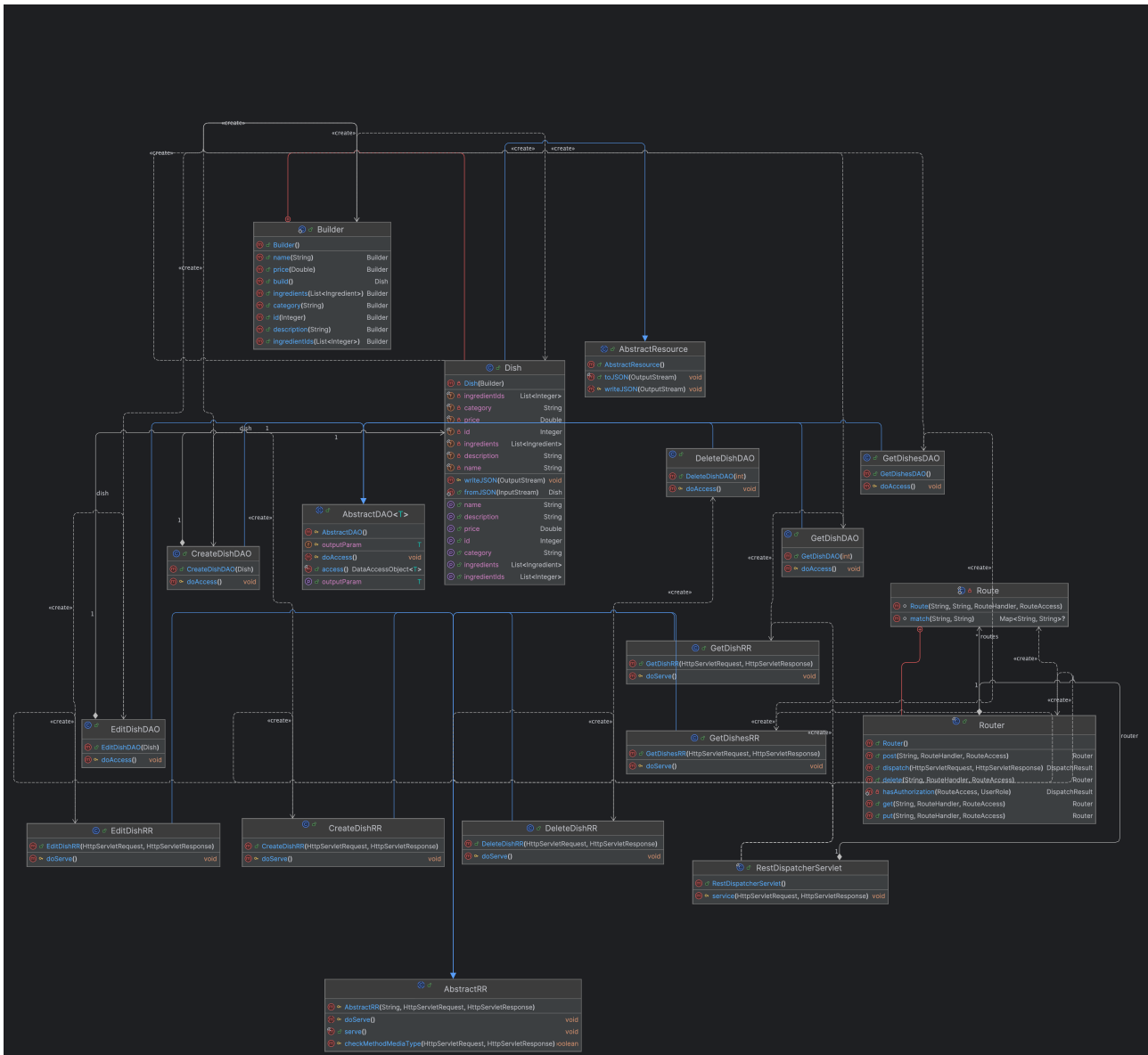
(a) Orders' view admin-side mockup page. The admin-side order view page provides detailed information about all orders.



(b) Orders' view staff-side mockup page. The staff-side orders view page shows the details needed by staff members to prepare or manage orders.

5 Business Logic Layer

5.1 Class Diagram



The business logic layer is organised into seven main packages: `dao`, `exception`, `filter`, `resources`, `rest`, `servlet` and `util`.

resources

Contains the classes that model the application data and are responsible for their own JSON serialisation. The root of the hierarchy is the `Resource` interface, which declares the single method `toJSON(OutputStream)`. `AbstractResource` provides a shared implementation backed by a Jackson `JsonFactory`, delegating the actual writing to the abstract method `writeJSON(OutputStream)`, which every concrete class must implement. The concrete resource classes are: `User` (uses the Builder pattern; password and hash are never serialised

to JSON), `Credentials` (login payload, never serialised), `Item`, `Ingredient`, `Order`, `Promotion`, and `Message` (error/info wrapper with optional `error_code` and `error_details` fields). Supporting types include the enums `UserRole` (`STAFF`, `CUSTOMER`, `ADMIN`), `OrderStatus` (`PENDING`, `COMPLETED`, `CANCELLED`), `Allergen` (14 EU allergens), and `ItemCategory` (predefined menu categories with display name and description). `ResourceList<T>` serialises any collection of `Resource` objects as a JSON array.

rest

Contains the per-endpoint controllers. `RestResource` is an interface declaring `serve()`. `AbstractRR` implements it and provides shared behaviour: MIME-type and HTTP method validation (checking `Accept` and `Content-Type` headers), structured error formatting via `Message` and per-thread logging via `LogContext`. Concrete subclasses override `doServe()`. Currently implemented: `RegisterUserRR` (handles `POST /rest/user/register`) and `AuthenticateUserRR` (handles `POST /rest/user/login`).

servlet

Contains the front-controller infrastructure. `RestDispatcherServlet` is a `@WebServlet` mapped to `/rest/*`; it sets the client IP in `LogContext`, delegates routing to a `Router` instance and handles any uncaught `Throwable` by returning `500`. `Router` matches HTTP method and path against registered `Route` objects, supports path parameters (e.g. `/rest/user/{id}`) and stores extracted parameters as request attributes. `RouteHandler` is a `@FunctionalInterface` used as the handler type, allowing checked exceptions to propagate.

filter

Contains the Jakarta Servlet filters applied to incoming HTTP requests. The package currently contains one class: `AuthFilter`, annotated with `@WebFilter(urlPatterns = "/dashboard/*")`. Its `doFilter()` method calls the private helper `extractTokenFromCookies()`, which iterates over the request cookies looking for the one whose name equals `JWTUtil.COOKIE_NAME` (`auth_token`). If no such cookie is found the method returns `null` and `doFilter()` immediately redirects to `/login`. If the cookie is found, `JWTUtil.verify(token)` is called. A `JWTVerificationException` (that means invalid signature or expired token) causes a redirect to `/login`. On successful verification the `user_id` claim is extracted from the `DecodedJWT` and stored as a request attribute. Then `chain.doFilter()` passes control to the next component in the pipeline.

dao

Contains the Data Access Objects. `DataAccessObject<T>` is an interface that declares `access()` and `getOutputParam()`. `AbstractDAO<T>` implements this interface and manages the full connection lifecycle. The latter acquires a connection from `ConnectionPoolSingleton` at construction time, calls `doAccess()` inside a try/catch block, commits or rolls back as needed and closes the connection. It also prevents double-calling via a `synchronized` accessed flag.

The concrete DAOs are organised in sub-packages by domain:

- **Root package:** `RegisterUserDAO` (inserts a new user with bcrypt hash, returns the generated `id`); `AuthenticateUserDAO` (looks up user by email, verifies password with bcrypt, returns `user_id` or `null`).
- **ingredient:** `NewIngredientDAO` (inserts an ingredient, converts allergen list to a SQL array); `GetIngredientDAO` (selects a single ingredient by id, deserialises the SQL allergen array); `GetIngredientsDAO` (selects all ingredients ordered by id); `EditIngredientDAO` (updates name, `is_frozen`, and allergen of an existing ingredient); `DeleteIngredientDAO` (deletes an ingredient by id, returns `Boolean` success flag).

- **item:** `CreateItemDAO` (inserts an item after validating non-null name, description, non-negative price, and non-null ingredients); `DeleteItemDAO` (deletes an item by id); `BindIngredientDAO` (inserts rows into `item_ingredients` for all ingredients of an item); `BindCategoryDAO` (inserts rows into `item_categories` for all categories of an item, validating that each `ItemCategory` has a non-empty name and description).
- **order:** `NewOrderDAO` (inserts an order, rejects null order or negative total price via a `NotValidOrder` runtime exception in the constructor); `GetOrderDAO` (retrieves a single order by id); `GetOrdersDAO` (retrieves all orders); `EditOrderDAO` (updates the status of an order); `DeleteOrderDAO` (deletes an order by id).
- **promotions:** `NewPromotionDAO` (inserts a promotion after validating it is not null and not expired); `CheckPromotionDAO` (looks up a promotion by code, throws `NotValidPromotionException` if not found or expired).

5.1.1 exception

Contains domain-specific checked exceptions used to signal validation failures at construction time or during DAO access: `MalformedJSONException` (malformed JSON request body, with multiple constructors), `NotValidIngredientException`, `NotValidItemException`, `NotValidCategoryException`, `NotValidPromotionException`, `ServiceException`. `NotValidOrder` is instead an unchecked `RuntimeException`, thrown in the `NewOrderDAO` constructor.

5.1.2 util

Contains cross-cutting utilities. `JWTUtil` is a static helper that initialises an HMAC-256 signer from the `jwt.secret` context parameter (read at startup by `AppContextListener`) and exposes `generateToken(int userId)` and `verify(String token)`; tokens contain the `user_id` claim and expire in 8 hours (28800 seconds). `Validator` provides static methods `isValidEmail` and `isValidPassword` using compiled regex patterns. `ConnectionPoolSingleton` looks up and exposes the container-managed JNDI `DataSource`. `AppContextListener` implements `ServletContextListener` and calls `JWTUtil.init()` at application startup. `ErrorCodes` centralises all error code string constants. `LogContext` wraps Log4j2's `ThreadContext` to store per-request context (IP address, action, resource) for structured logging. `Actions` holds string constants for the action names used in logging.

5.1.3 test

The project includes a unit test class `JWTUtilTest`, which verifies that a generated token contains the expected `user_id` claim, that a token signed with a different secret is rejected, and that a malformed string is rejected by `JWTUtil.verify()`.

5.2 Sequence Diagram

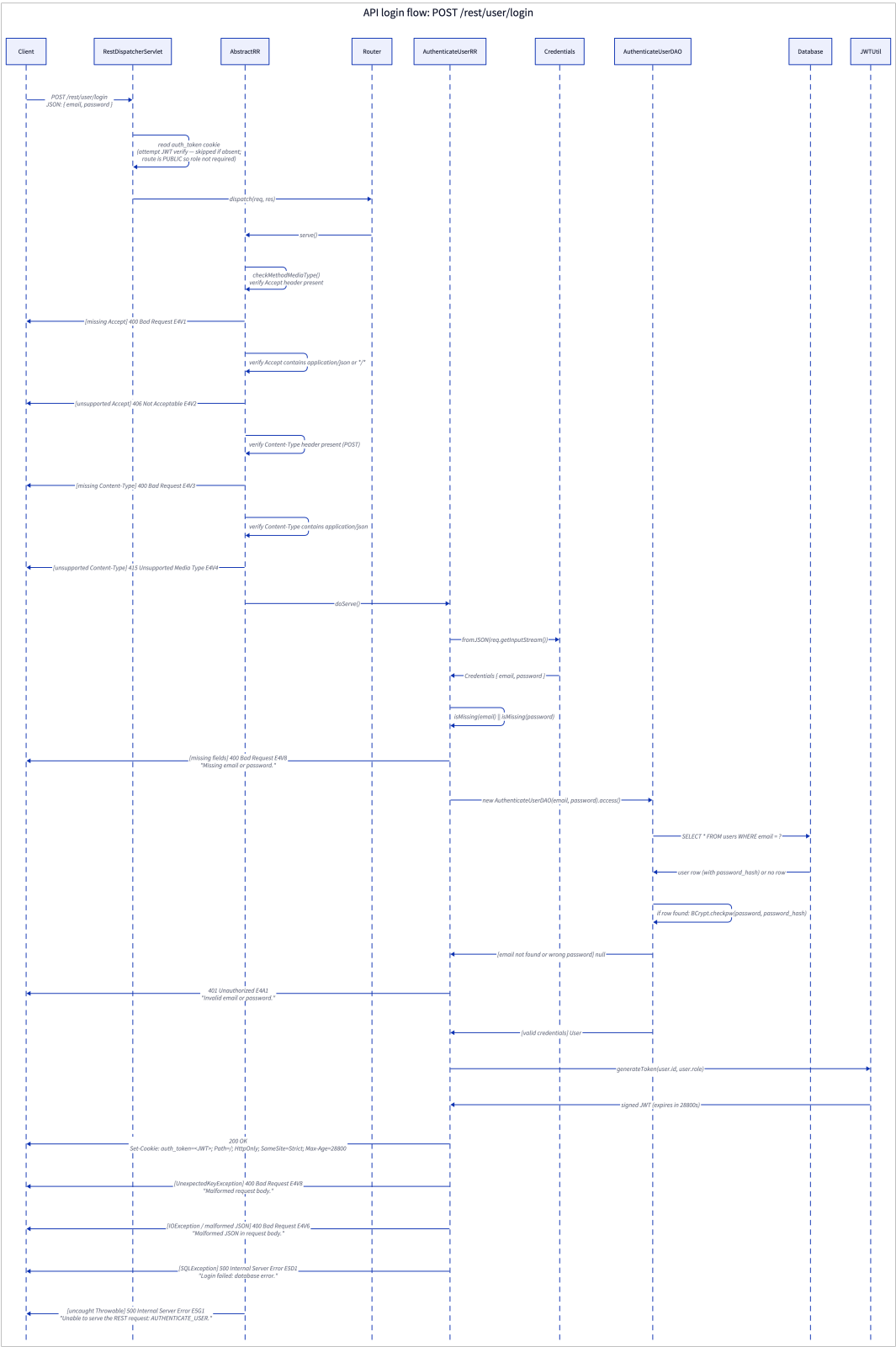


Figure 16: Home page's mockup

The following three examples are representative of how the application processes HTTP requests. The first shows user registration, the second shows user login, and the third shows how an authenticated request to the dashboard is handled by the `AuthFilter`.

User Registration - POST `/rest/user/register`

When a client sends a `POST` request to `/rest/user/register`, the Tomcat container receives it and forwards it to `RestDispatcherServlet`, which is the single entry point for all `/rest/*` requests. The dispatcher sets the client IP in `LogContext`, then uses its internal `Router` to match the method and path against registered routes and instantiates `RegisterUserRR`.

`RegisterUserRR.doServe()` first invokes `AbstractRR.checkMethodMediaType()` to verify that the `Accept` header contains `application/json` and that the `Content-Type` header is also `application/json`; if either check fails, an appropriate error `Message` is returned immediately with the corresponding error code (`E4V1–E4V5`). It then deserialises the request body into a `User` object via `User.fromJSON()`, which throws `MalformedJSONException` if an unexpected field is encountered. If any required field (username, email, password, name, surname, phone number) is missing or blank, a `400 Bad Request` response is returned with error code `E4V8`. If the email does not match the expected pattern or the password does not satisfy the security policy, the same error code is returned with a descriptive message.

If validation passes, a `RegisterUserDAO` is instantiated with the validated `User` object and `access()` is called. `AbstractDAO.access()` acquires a connection from `ConnectionPoolSingleton` and delegates to `RegisterUserDAO.doAccess()`, which hashes the plain-text password with `BCrypt.hashpw()` and executes an `INSERT` returning the generated user id. If the email, username, or phone number already exists, PostgreSQL raises a `23505` unique constraint violation; `RegisterUserRR` catches this and returns `409 Conflict` with error code `E4R3`. Any other `SQLException` yields `500 Internal Server Error` with error code `E5D1`. On success, `RegisterUserRR` sets the status to `201 Created` and serialises a `User` object containing only the new user's id.

User Authentication - POST `/rest/user/login`

When a client sends a `POST` request to `/rest/user/login`, the dispatcher instantiates `AuthenticateUserRR`. After the standard MIME-type checks by `checkMethodMediaType()`, the handler deserialises the request body into a `Credentials` object via `Credentials.fromJSON()`, verifying that both `email` and `password` fields are present and non-blank. A `MalformedJSONException` is thrown if unexpected fields are found.

An `AuthenticateUserDAO` is instantiated with the provided email and password and `access()` is called. `doAccess()` executes `SELECT id, password_hash FROM users WHERE email = ?`. If no row is found, `outputParam` remains `null`. If a row is found, `BCrypt.checkpw()` verifies the submitted password against the stored hash; on match, `outputParam` is set to the user's id. A `null` output param causes `AuthenticateUserRR` to return `401 Unauthorized` with error code `E4A1`. On success, `JWTUtil.generateToken(userId)` produces a signed HMAC-256 JWT containing the `user_id` claim and an 8-hour expiry. The token is attached to the response via `Set-Cookie` with the flags `HttpOnly` and `SameSite=Strict`, and the status is set to `200 OK`.

Authenticated Dashboard Request - GET `/dashboard/*`

When an authenticated user sends any request to a URL matching `/dashboard/*`, the Tomcat container intercepts it with the `AuthFilter` before reaching any servlet. The filter calls `extractTokenFromCookies()` to find the cookie named `auth_token` among the request cookies. If no such cookie exists, the filter immediately calls `res.sendRedirect("/login")` and the chain is not invoked.

If the cookie is found, `JWTUtil.verify(token)` is called. This method uses the pre-initialised HMAC-256 verifier to check both the signature and the expiry time of the token. If verification throws a

`JWTVerificationException` (invalid signature or expired token), the filter redirects to `/login` with error code `E4A3`. If verification succeeds, the `user_id` claim is extracted from the decoded JWT and stored as a request attribute (`req.setAttribute("user_id", userId)`). The filter then calls `chain.doFilter(request, response)`, passing the enriched request to the next component in the pipeline, which can read the `user_id` attribute to identify the authenticated user without re-querying the database.

5.3 REST API Summary

All REST resources are identified through the prefix `/rest/`, which is omitted in the table below. No servlet filter is mapped to `/rest/`; REST authentication and authorization are enforced inside `RestDispatcherServlet`, which reads the `auth_token` `HttpOnly` cookie, verifies the JWT, sets the `user_id` and `user_role` request attributes, and applies per-route access rules.

- Dispatcher Access Guard (DAG): effective REST access-control layer inside `RestDispatcherServlet`, applied to all `/rest/` routes. `DAG-P` = `PUBLIC`, `DAG-A` = `AUTHENTICATED`, `DAG-SA` = `STAFF_OR_ADMIN`, `DAG-ADM` = `ADMIN_ONLY`. Missing, invalid, or expired JWTs are treated as unauthenticated and protected routes return `401` with `E4A2`; insufficient roles return `403` with `E4A4`.
- DashboardFilter (DF): protects `/dashboard/` by validating the `auth_token` `HttpOnly` cookie. Redirects to `/login` if the token is missing or invalid; otherwise sets the `user_id` request attribute. This filter does not apply to `/rest/`.
- AuthFlowFilter (AFF): protects `/login` and `/register` by redirecting already-authenticated users to `/dashboard`. This filter does not apply to `/rest/`.

URI	Method	Description	Filter
<code>/user/login</code>	<code>POST</code>	Authenticates a user and issues a JWT cookie	<code>DAG-P</code>
<code>/user/logout</code>	<code>POST</code>	Expires the JWT cookie	<code>DAG-A</code>
<code>/user/register</code>	<code>POST</code>	Registers a new customer account	<code>DAG-P</code>
<code>/user</code>	<code>GET</code>	Returns the authenticated user profile	<code>DAG-A</code>
<code>/user</code>	<code>PUT</code>	Updates the authenticated user profile	<code>DAG-A</code>
<code>/user</code>	<code>DELETE</code>	Deletes the authenticated user account	<code>DAG-A</code>
<code>/category</code>	<code>POST</code>	Creates a category	<code>DAG-ADM</code>
<code>/category</code>	<code>GET</code>	Returns all categories	<code>DAG-P</code>
<code>/category/{category_name}</code>	<code>DELETE</code>	Deletes a category by name	<code>DAG-ADM</code>
<code>/ingredient</code>	<code>POST</code>	Creates an ingredient	<code>DAG-SA</code>
<code>/ingredient</code>	<code>GET</code>	Returns all ingredients	<code>DAG-P</code>
<code>/ingredient/{ingredient_id}</code>	<code>GET</code>	Returns one ingredient	<code>DAG-P</code>
<code>/ingredient</code>	<code>PUT</code>	Updates an ingredient	<code>DAG-SA</code>
<code>/ingredient/{ingredient_id}</code>	<code>DELETE</code>	Deletes an ingredient	<code>DAG-SA</code>
<code>/dish</code>	<code>POST</code>	Creates a dish	<code>DAG-SA</code>
<code>/dish</code>	<code>GET</code>	Returns all dishes	<code>DAG-P</code>
<code>/dish/{dish_id}</code>	<code>GET</code>	Returns one dish	<code>DAG-P</code>
<code>/dish</code>	<code>PUT</code>	Updates a dish	<code>DAG-SA</code>
<code>/dish/{dish_id}</code>	<code>DELETE</code>	Deletes a dish	<code>DAG-SA</code>
<code>/order</code>	<code>GET</code>	Returns all orders	<code>DAG-SA</code>
<code>/order</code>	<code>POST</code>	Creates an order	<code>DAG-A</code>
<code>/order/user</code>	<code>GET</code>	Returns all orders of the authenticated user	<code>DAG-A</code>
<code>/order/{order_id}</code>	<code>GET</code>	Returns one order	<code>DAG-A</code>
<code>/order/{order_id}/{order_status}</code>	<code>PUT</code>	Updates an order status	<code>DAG-SA</code>

URI	Method	Description	Filter
/order/{order_id}/	DELETE	Deletes an order	DAG-ADM
/promotion	POST	Creates a promotion	DAG-ADM
/promotion	GET	Returns all promotions	DAG-SA
/promotion/{promotion_code}	GET	Returns a promotion by code	DAG-P
/promotion/{promotion_code}/usage	GET	Returns whether the authenticated user already used the promotion	DAG-A

Table 2: REST API summary

5.4 REST Error Codes

Listed below are the error codes defined for the application. Each code uniquely identifies a specific error condition and follows the format `E[class][category][index]`, where class is the HTTP status class, category identifies the error domain, and index is a sequential number within the category. The defined categories are: **A** (authentication), **V** (validation/media type), **R** (resource), **D** (database), and **G** (generic server error).

Error Code	HTTP Status Code	Description
E4A1	401 Unauthorized	Invalid login credentials: wrong e-mail or password.
E4A2	401 Unauthorized	Missing authentication cookie. In current REST routing, invalid or expired JWTs are also collapsed to this code because the dispatcher treats them as unauthenticated.
E4A3	401 Unauthorized	Invalid or expired access token. Defined in <code>ErrorCodes</code> , but not emitted by the current REST dispatcher.
E4A4	403 Forbidden	Forbidden operation due to insufficient role.
E4V1	400 Bad Request	Output media type not specified: missing <code>Accept</code> header.
E4V2	406 Not Acceptable	Unsupported output media type: <code>Accept</code> does not include <code>application/json</code> and is not exactly <code>/*/*</code> .
E4V3	400 Bad Request	Input media type not specified: missing <code>Content-Type</code> header on <code>POST</code> or <code>PUT</code> .
E4V4	415 Unsupported Media Type	Unsupported input media type: <code>Content-Type</code> does not contain <code>application/json</code> .
E4V5	405 Method Not Allowed	Unsupported operation. Defined in <code>AbstractRR</code> ; with the current router, unmatched methods normally fall through to <code>E4R1</code> instead.
E4V6	400 Bad Request	Invalid input parameter.
E4V7	400 Bad Request	Wrong URI format. Defined in <code>ErrorCodes</code> , but not used by the current REST routes.
E4V8	400 Bad Request	Wrong resource provided: malformed request body or semantically invalid representation.
E4R1	404 Not Found	Unknown resource requested: no configured route matched the request URI.
E4R2	404 Not Found	Resource not found.
E4R3	409 Conflict	Resource already exists.
E4R4	409 Conflict	Resource still referenced by other resources. Defined in <code>ErrorCodes</code> , but not used by the current REST handlers.
E5D1	500 Internal Server Error	Unexpected database error. Some endpoints also use this code for conflicts or foreign-key/check-constraint failures instead of mapping them to a 4xx code.
E5G1	500 Internal Server Error	Unexpected generic server error. This is the fallback used by <code>AbstractRR</code> or <code>RestDispatcherServlet</code> for uncaught exceptions.

Table 3: List of the REST error codes

5.5 REST API Details

All error responses are serialized through `Message.toJSON()` and therefore use the envelope `{"message":{"message":...,"error_code":...,"error_details":...}}`; `error_code` and `error_details` are omitted when null. `Accept` is mandatory for every documented endpoint and must include `application/json` or be exactly `*/*`. For `POST` and `PUT`, `Content-Type` is also mandatory and must contain `application/json`; this includes `POST /user/logout` and `PUT /order/{order_id}/{order_status}`, even though those handlers do not read a request body.

Common globally enforced REST errors are:

- `400 Bad Request`, `E4V1` - inner message `Output media type not specified.`, inner details `Accept request header missing.`
- `406 Not Acceptable`, `E4V2` - inner message `Unsupported output media type. Resources are represented on`, inner details `Requested representation is <Accept>.`
- `400 Bad Request`, `E4V3` - inner message `Input media type not specified.`, inner details `Content-Type request header missing.`
- `415 Unsupported Media Type`, `E4V4` - inner message `Unsupported input media type. Resources are represe`, inner details `Submitted representation is <Content-Type>.`
- Protected-route dispatcher errors:
 - `401 Unauthorized`, `E4A2` - inner message `Authentication required.` `403 Forbidden`, `E4A4` - inner message `You do not have permission to perform this operation.`
- Unmatched-route dispatcher error:
 - `404 Not Found`, `E4R1` - inner message `Unknown resource requested.`, inner details `Requested resource is <request URI>.`
- Generic fallback:
 - `500 Internal Server Error`, `E5G1` - either `Unexpected error.` from `RestDispatcherServlet` or `Unable to serve the REST request: <ACTION>.` from `AbstractRRR`, with dynamic `error_details`.

Register User

Creates a new user account. The DAO always persists role `CUSTOMER`.

- **URL:** `/user/register`
- **Method:** `POST`
- **URL Parameters:** `None`
- **Query Parameters:** `None`
- **Headers:** `Content-Type: application/json; Accept: application/json`
- **Cookies:** `None`
- **Data Parameters:**

```
1 {
2   "username": "newcustomer",
3   "email": "newcustomer@example.com",
4   "password": "Secret1!",
5   "name": "Giulia",
6   "surname": "Rossi",
7   "phone_number": "3331234567"
8 }
```

Required fields are `username`, `email`, `password`, `name`, `surname`, and `phone_number`; blank values are rejected for all of them. `email` must match `^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$`. `password` must be 8-16 characters long and contain at least one lowercase letter, one uppercase letter, one digit, and one special character. The shared `User.fromJSON()` parser also accepts `id` and `role`, but this endpoint ignores them on success: the database generates the id and `RegisterUserDAO` always inserts `CUSTOMER`. An invalid `role` enum value is not explicitly handled and falls through to `E5G1`.

- **Success Response:** `201 Created`

Content:

```
1 {
2   "id": 6
3 }
```

- **Error Responses:**

- Common header errors: `E4V1`, `E4V2`, `E4V3`, `E4V4`.
- `400 Bad Request`, `E4V8` - inner message `Missing required fields.`, inner details `Fields 'username', 'email', 'password', 'name', 'surname', and 'phone_number' are required.`
- `400 Bad Request`, `E4V8` - inner message `Invalid email address.`, inner details `The provided email address is not valid.`
- `400 Bad Request`, `E4V8` - inner message `Invalid password.`, inner details `Password must be 8-16 characters and contain at least one uppercase letter, one lowercase letter, one digit, and one special character.`
- `400 Bad Request`, `E4V8` - inner message `Malformed request body.`, inner details are the parser message or `Unexpected key in JSON: <field>`.
- `409 Conflict`, `E4R3` - inner message `Email, username, or phone number already in use.`
- `500 Internal Server Error`, `E5D1` - inner message `Registration failed: database error.`, inner details are the SQL exception message.
- `500 Internal Server Error`, `E5G1` - uncaught runtime failures, including invalid `role` enum values accepted by the shared parser.

- **Sample Request Body:**

```
1 {
2   "username": "newcustomer",
3   "email": "newcustomer@example.com",
4   "password": "Secret1!",
5   "name": "Giulia",
6   "surname": "Rossi",
7   "phone_number": "3331234567"
8 }
```

Authenticate User

Authenticates an existing user and issues a signed JWT in the `auth_token` `HttpOnly` cookie.

- **URL:** `/user/login`
- **Method:** `POST`
- **URL Parameters:** `None`
- **Query Parameters:** `None`
- **Headers:** `Content-Type: application/json; Accept: application/json`
- **Cookies:** `None`
- **Data Parameters:**

```
1 {
2   "email": "merja@test.com",
3   "password": "@UniPadova26"
4 }
```

Only `email` and `password` are parsed. No controller-level e-mail format validation is performed here; the credentials are passed directly to `AuthenticateUserDAO`. On success, the JWT contains the claims `user_id` and `user_role`, expires after 8 hours (28800 seconds), and is sent as `Set-Cookie: auth_token=<JWT>; Path=/; HttpOnly; SameSite=Strict; Max-Age=28800`. No `Secure` attribute is added in code.

- **Success Response:** 200 OK

Content: `None` Additional header: `Set-Cookie: auth_token=<JWT>; Path=/; HttpOnly; SameSite=Strict; Max-`

- **Error Responses:**

- Common header errors: `E4V1`, `E4V2`, `E4V3`, `E4V4`.
- 400 Bad Request, `E4V8` - inner message `Missing email or password.`, inner details `Fields 'email' and 'password' are required.`
- 400 Bad Request, `E4V8` - inner message `Malformed request body.`, inner details are `Unexpected key in JSON: <field>.`
- 400 Bad Request, `E4V6` - inner message `Malformed JSON in request body.`, inner details are the Jackson parser message.
- 401 Unauthorized, `E4A1` - inner message `Invalid email or password.`
- 500 Internal Server Error, `E5D1` - inner message `Login failed: database error.`, inner details are the SQL exception message.

- **Sample Request Body:**

```
1 {  
2   "email": "merja@test.com",  
3   "password": "@UniPadova26"  
4 }
```

Logout User

Expires the JWT cookie.

- **URL:** `/user/logout`
- **Method:** `POST`
- **URL Parameters:** `None`
- **Query Parameters:** `None`
- **Headers:** `Content-Type: application/json; Accept: application/json`
- **Cookies:** `auth_token` `HttpOnly` cookie containing a valid JWT
- **Data Parameters:** `None`

Although the handler does not read a body, the shared `AbstractRR` logic still requires `Content-Type: application/json` because the route uses `POST`.

- **Success Response:** 204 No Content

Content: `None` Additional header: `Set-Cookie: auth_token=; Path=/; HttpOnly; SameSite=Strict; Max-Age=0`

- **Error Responses:**

- Common auth error: `401 Unauthorized`, `E4A2`. Invalid or expired JWTs are not mapped to `E4A3`; they are treated as unauthenticated and produce `E4A2`.
- Common header errors: `E4V1`, `E4V2`, `E4V3`, `E4V4`.

Get Current User

Returns the profile of the authenticated user identified by the JWT `user_id` claim.

- **URL:** `/user`
- **Method:** `GET`
- **URL Parameters:** `None`
- **Query Parameters:** `None`

- **Headers:** `Accept: application/json`
- **Cookies:** `auth_token` HttpOnly cookie containing a valid JWT
- **Data Parameters:** `None`

The endpoint does not accept a path id. It always resolves the current user from the dispatcher-set `user_id` request attribute.

- **Success Response:** `200 OK`

Content:

```
1 {
2   "id": 4,
3   "username": "gpadoan",
4   "email": "padoan@test.com",
5   "name": "Giancarlo",
6   "surname": "Padoan",
7   "phone_number": "1234567",
8   "role": "CUSTOMER"
9 }
```

- **Error Responses:**

- Common auth error: `401 Unauthorized`, `E4A2`.
- Common header errors: `E4V1`, `E4V2`.
- `404 Not Found`, `E4R2` - inner message `User with id 4 not found.` when the JWT user id no longer exists in the database.
- `400 Bad Request`, `E4V6` - inner message `Invalid user ID: <value>` if the dispatcher-set `user_id` attribute is not numeric.
- `500 Internal Server Error`, `E5D1` - inner message `Database error while retrieving user.`, inner details are the SQL exception message.

Edit Current User

Updates the authenticated user's profile.

- **URL:** `/user`
- **Method:** `PUT`
- **URL Parameters:** `None`
- **Query Parameters:** `None`
- **Headers:** `Content-Type: application/json`; `Accept: application/json`
- **Cookies:** `auth_token` HttpOnly cookie containing a valid JWT
- **Data Parameters:**

```
1 {
2   "name": "Giancarlo",
3   "surname": "Padoan",
4   "email": "giancarlo.padoan@example.com",
5   "phone_number": "3339876543"
6 }
```

The endpoint uses the JWT `user_id`; it does not accept a target user id in the URL. Only `name`, `surname`, `email`, and `phone_number` are actually required by the handler, and it checks only for `null`, not blank strings. No controller-level e-mail-format validation is performed here. The shared `User.fromJSON()` parser also accepts `id`, `username`, `role`, and `password`, but this handler ignores them on success; invalid `role` enum values and malformed JSON syntax are not explicitly caught and fall through to `E5G1`.

- **Success Response:** `200 OK`

Content:

```
1 {
2   "id": 4,
```

```

3  "username": "gpadoan",
4  "email": "giancarlo.padoan@example.com",
5  "name": "Giancarlo",
6  "surname": "Padoan",
7  "phone_number": "3339876543",
8  "role": "CUSTOMER"
9  }

```

- **Error Responses:**

- Common auth error: 401 Unauthorized, E4A2.
- Common header errors: E4V1, E4V2, E4V3, E4V4.
- 400 Bad Request, E4V6 - inner message Missing required fields: name, surname, email and phone_number.
- 400 Bad Request, E4V8 - inner message Malformed JSON in request body., inner details are Unexpected key in JSON: <field>.
- 404 Not Found, E4R2 - inner message User with id 4 not found.
- 409 Conflict, E4R3 - inner message Email or phone number already in use.
- 500 Internal Server Error, E5D1 - inner message Database error while updating user., inner details are the SQL exception message.
- 500 Internal Server Error, E5G1 - malformed JSON syntax and invalid role enum values fall through to Unable to serve the REST request: EDIT_USER.

- **Sample Request Body:**

```

1  {
2    "name": "Giancarlo",
3    "surname": "Padoan",
4    "email": "giancarlo.padoan@example.com",
5    "phone_number": "3339876543"
6  }

```

Delete Current User

Deletes the authenticated user's account.

- **URL:** /user
- **Method:** DELETE
- **URL Parameters:** None
- **Query Parameters:** None
- **Headers:** Accept: application/json
- **Cookies:** auth_token HttpOnly cookie containing a valid JWT
- **Data Parameters:** None

The endpoint uses the JWT user_id. Because the schema defines orders.user_id with ON DELETE CASCADE, deleting a user also deletes that user's orders. The handler does not clear the auth cookie after deletion.

- **Success Response:** 200 OK

Content:

```

1  {
2    "id": 4,
3    "username": "gpadoan",
4    "email": "padoan@test.com",
5    "name": "Giancarlo",
6    "surname": "Padoan",
7    "phone_number": "1234567",
8    "role": "CUSTOMER"
9  }

```

- **Error Responses:**

- Common auth error: 401 Unauthorized, E4A2.
- Common header errors: E4V1, E4V2.
- 404 Not Found, E4R2 - inner message User with id 4 not found.
- 400 Bad Request, E4V6 - inner message Invalid user ID: <value> if the dispatcher-set user_id is not numeric.
- 500 Internal Server Error, E5D1 - inner message Database error while deleting user., inner details are the SQL exception message.

Create Category

Creates a new category.

- **URL:** /category
- **Method:** POST
- **URL Parameters:** None
- **Query Parameters:** None
- **Headers:** Content-Type: application/json; Accept: application/json
- **Cookies:** auth_token HttpOnly cookie containing a valid JWT with role ADMIN
- **Data Parameters:**

```
1 {
2   "name": "Pizze"
3 }
```

Only the `name` field is parsed. `name` is required and blank values are rejected. Unexpected fields are mapped to E4V8. Malformed JSON syntax is not explicitly caught here and therefore falls through to E5G1.

- **Success Response:** 201 Created
Content:

```
1 {
2   "name": "Pizze"
3 }
```

- **Error Responses:**
 - Common auth errors: 401 Unauthorized, E4A2; 403 Forbidden, E4A4.
 - Common header errors: E4V1, E4V2, E4V3, E4V4.
 - 400 Bad Request, E4V6 - inner message Missing required field: name must be provided.
 - 400 Bad Request, E4V8 - inner message Malformed JSON in request body., inner details are Unexpected field: <field>.
 - 409 Conflict, E5D1 - inner message Category already exists., inner details are the database exception message. The status is 409, but the code is the database code E5D1, not E4R3.
 - 500 Internal Server Error, E5D1 - inner message Database error while creating category., inner details are the SQL exception message.
 - 500 Internal Server Error, E5G1 - malformed JSON syntax falls through to Unable to serve the REST request: CREATE_CATEGORY.
- **Sample Request Body:**

```
1 {
2   "name": "Pizze"
3 }
```

Get All Categories

Returns all categories.

- **URL:** /category

- **Method:** GET
- **URL Parameters:** None
- **Query Parameters:** None
- **Headers:** Accept: application/json
- **Cookies:** None
- **Data Parameters:** None
- **Success Response:** 200 OK

Content:

```

1 {
2   "resource-list": [
3     { "name": "Antipasti" },
4     { "name": "Primi" },
5     { "name": "Secondi" }
6   ]
7 }
```

- **Error Responses:**
 - Common header errors: E4V1, E4V2.
 - 500 Internal Server Error, E5D1 - inner message Database error while retrieving categories., inner details are the SQL exception message.

Delete Category

Deletes a category by name.

- **URL:** /category/{category_name}
- **Method:** DELETE
- **URL Parameters:** category_name - category primary key
- **Query Parameters:** None
- **Headers:** Accept: application/json
- **Cookies:** auth_token HttpOnly cookie containing a valid JWT with role ADMIN
- **Data Parameters:** None

The path parameter is matched as a raw route segment. Database matching is exact because the DAO executes `DELETE FROM categories WHERE name = -`.

- **Success Response:** 200 OK

Content:

```

1 {
2   "name": "Pizze"
3 }
```

- **Error Responses:**
 - Common auth errors: 401 Unauthorized, E4A2; 403 Forbidden, E4A4.
 - Common header errors: E4V1, E4V2.
 - 404 Not Found, E4R2 - inner message Category 'Pizze' not found.
 - 409 Conflict, E5D1 - inner message Cannot delete category: it is referenced by existing dishes., inner details are the SQL exception message. The schema conflict is detected via SQL state 23503, but the handler still emits E5D1.
 - 500 Internal Server Error, E5D1 - inner message Database error while deleting category., inner details are the SQL exception message.

Create Ingredient

Creates an ingredient.

- **URL:** /ingredient

- **Method:** POST
- **URL Parameters:** None
- **Query Parameters:** None
- **Headers:** Content-Type: application/json; Accept: application/json
- **Cookies:** auth_token HttpOnly cookie containing a valid JWT with role STAFF or ADMIN
- **Data Parameters:**

```

1 {
2   "name": "Origano",
3   "allergens": [],
4   "is_frozen": false
5 }

```

The parser accepts `id`, `name`, `allergens`, and `is_frozen`. The handler requires only `name`, checked for `null` (not blank). `is_frozen` and `allergens` are optional; `allergens` may be omitted, set to `null`, or be an empty array. Valid allergen values are the lowercase enum literals `cereals`, `crustaceans`, `eggs`, `fish`, `peanuts`, `soybeans`, `milk`, `nuts`, `celery`, `mustard`, `sesame_seeds`, `sulphur_dioxide_and_sulphites`, `lupin`, and `molluscs`. Invalid allergen enum values are not explicitly handled and fall through to E5G1.

- **Success Response:** 201 Created
Content:

```

1 {
2   "id": 6,
3   "name": "Origano",
4   "allergens": [],
5   "is_frozen": false
6 }

```

- **Error Responses:**

- Common auth errors: 401 Unauthorized, E4A2; 403 Forbidden, E4A4.
- Common header errors: E4V1, E4V2, E4V3, E4V4.
- 400 Bad Request, E4V6 - inner message Missing required fields: name and isFrozen must be provided when name is null.
- 400 Bad Request, E4V8 - inner message Malformed JSON in request body., inner details are Unexpected field: <field>.
- 500 Internal Server Error, E5D1 - inner message Failed to create ingredient. if the DAO returns no row.
- 500 Internal Server Error, E5D1 - inner message Database error while creating ingredient., inner details are the SQL exception message.
- 500 Internal Server Error, E5G1 - malformed JSON syntax or invalid allergen enum values fall through to Unable to serve the REST request: CREATE_INGREDIENT.

- **Sample Request Body:**

```

1 {
2   "name": "Origano",
3   "allergens": [],
4   "is_frozen": false
5 }

```

Get All Ingredients

Returns all ingredients.

- **URL:** /ingredient
- **Method:** GET
- **URL Parameters:** None

- **Query Parameters:** None
- **Headers:** Accept: application/json
- **Cookies:** None
- **Data Parameters:** None

The read DAOs use `ResultSet#getBoolean("is_frozen")` without `wasNull()`, so a database `NULL` in `ingredients.is_frozen` would be serialized as `false`.

- **Success Response:** 200 OK

Content:

```

1 {
2   "resource-list": [
3     {
4       "id": 1,
5       "name": "Pomodoro San Marzano",
6       "allergens": [],
7       "is_frozen": false
8     },
9     {
10      "id": 2,
11      "name": "Mozzarella di Bufala",
12      "allergens": ["milk"],
13      "is_frozen": false
14    }
15  ]
16 }
```

- **Error Responses:**

- Common header errors: E4V1, E4V2.
- 500 Internal Server Error, E5D1 - inner message Database error while retrieving ingredients, with no explicit error_details.

Get Ingredient

Returns one ingredient by id.

- **URL:** /ingredient/{ingredient_id}
- **Method:** GET
- **URL Parameters:** ingredient_id - integer ingredient id
- **Query Parameters:** None
- **Headers:** Accept: application/json
- **Cookies:** None
- **Data Parameters:** None
- **Success Response:** 200 OK

Content:

```

1 {
2   "id": 2,
3   "name": "Mozzarella di Bufala",
4   "allergens": ["milk"],
5   "is_frozen": false
6 }
```

- **Error Responses:**

- Common header errors: E4V1, E4V2.
- 400 Bad Request, E4V6 - inner message Ingredient id must be a valid integer
- 404 Not Found, E4R2 - inner message Ingredient 2 not found in database
- 500 Internal Server Error, E5D1 - inner message Database error while retrieving ingredient, with no explicit error_details.

Edit Ingredient

Updates an ingredient.

- **URL:** `/ingredient`
- **Method:** `PUT`
- **URL Parameters:** `None`
- **Query Parameters:** `None`
- **Headers:** `Content-Type: application/json; Accept: application/json`
- **Cookies:** `auth_token` HttpOnly cookie containing a valid JWT with role `STAFF` or `ADMIN`
- **Data Parameters:**

```
1 {
2   "id": 2,
3   "name": "Mozzarella di Bufala DOP",
4   "allergens": ["milk"],
5   "is_frozen": false
6 }
```

The handler requires `id`, `name`, and `is_frozen`; unlike create, it also rejects blank `name`. `allergens` remains optional. Invalid allergen enum values and malformed JSON syntax are not explicitly mapped and fall through to `E5G1`.

- **Success Response:** `200 OK`

Content:

```
1 {
2   "id": 2,
3   "name": "Mozzarella di Bufala DOP",
4   "allergens": ["milk"],
5   "is_frozen": false
6 }
```

- **Error Responses:**

- Common auth errors: `401 Unauthorized`, `E4A2`; `403 Forbidden`, `E4A4`.
- Common header errors: `E4V1`, `E4V2`, `E4V3`, `E4V4`.
- `400 Bad Request`, `E4V6` - inner message `Missing required fields: name and isFrozen must be provided`
- `400 Bad Request`, `E4V8` - inner message `Unexpected key in JSON: Unexpected field: <field>`
- `404 Not Found`, `E4R2` - inner message `Ingredient with id 2 not found.`
- `500 Internal Server Error`, `E5D1` - inner message `Database error while editing ingredient.`, inner details are the SQL exception message.
- `500 Internal Server Error`, `E5G1` - malformed JSON syntax or invalid allergen enum values fall through to `Unable to serve the REST request: EDIT_INGREDIENT.`

- **Sample Request Body:**

```
1 {
2   "id": 2,
3   "name": "Mozzarella di Bufala DOP",
4   "allergens": ["milk"],
5   "is_frozen": false
6 }
```

Delete Ingredient

Deletes an ingredient by id.

- **URL:** `/ingredient/{ingredient_id}`
- **Method:** `DELETE`
- **URL Parameters:** `ingredient_id` - integer ingredient id
- **Query Parameters:** `None`

- **Headers:** `Accept: application/json`
- **Cookies:** `auth_token` HttpOnly cookie containing a valid JWT with role `STAFF` or `ADMIN`
- **Data Parameters:** `None`
The schema defines `dish_ingredients.ingredient_id` with `ON DELETE CASCADE`, so deleting an ingredient also deletes its dish bindings.
- **Success Response:** `200 OK`
Content:


```

1 {
2   "id": 6,
3   "name": "Origano",
4   "allergens": [],
5   "is_frozen": false
6 }
```
- **Error Responses:**
 - Common auth errors: `401 Unauthorized`, `E4A2`; `403 Forbidden`, `E4A4`.
 - Common header errors: `E4V1`, `E4V2`.
 - `400 Bad Request`, `E4V6` - inner message `Invalid ingredient id format: /6` or the actual `req.getPathInfo()` value, inner details are the `NumberFormatException` message.
 - `404 Not Found`, `E4R2` - inner message `Ingredient with id 6 not found`
 - `500 Internal Server Error`, `E5D1` - inner message `Unexpected database error: no. <sqlErrorCode>`, inner details are the SQL exception message.

Create Dish

Creates a dish and its ingredient bindings.

- **URL:** `/dish`
- **Method:** `POST`
- **URL Parameters:** `None`
- **Query Parameters:** `None`
- **Headers:** `Content-Type: application/json`; `Accept: application/json`
- **Cookies:** `auth_token` HttpOnly cookie containing a valid JWT with role `STAFF` or `ADMIN`
- **Data Parameters:**

```

1 {
2   "name": "Margherita",
3   "description": "Pizza con pomodoro, mozzarella e basilico",
4   "price": 8.5,
5   "ingredient_ids": [1, 2, 3],
6   "category": "Pizze"
7 }
```

Required fields are `name`, `price`, `ingredient_ids`, and `category`. `name` must be non-blank, `price` must be strictly greater than 0, and `ingredient_ids` must be a non-empty array. `description` is optional. The parser also accepts `id`, but the create logic ignores it. On success, the response contains nested `ingredients`, not `ingredient_ids`. Invalid category names, invalid ingredient ids, and duplicate ingredient ids in the submitted array surface as database errors (`E5D1`), not as explicit 4xx validation errors.

- **Success Response:** `201 Created`
Content:

```

1 {
2   "id": 6,
3   "name": "Margherita",
4   "description": "Pizza con pomodoro, mozzarella e basilico",
5   "price": 8.5,
6   "ingredients": [
```

```

7      {
8        "id": 1,
9        "name": "Pomodoro San Marzano",
10       "allergens": [],
11       "is_frozen": false
12     },
13     {
14       "id": 2,
15       "name": "Mozzarella di Bufala",
16       "allergens": ["milk"],
17       "is_frozen": false
18     },
19     {
20       "id": 3,
21       "name": "Basilico fresco",
22       "allergens": [],
23       "is_frozen": false
24     }
25   ],
26   "category": "Pizze"
27 }

```

- **Error Responses:**

- Common auth errors: 401 Unauthorized, E4A2; 403 Forbidden, E4A4.
- Common header errors: E4V1, E4V2, E4V3, E4V4.
- 400 Bad Request, E4V6 - inner message Missing or invalid fields
- 400 Bad Request, E4V8 - inner message Malformed JSON in request body., inner details are Unexpected field: <field>.
- 500 Internal Server Error, E5D1 - inner message Failed to create dish. if the DAO returns no row.
- 500 Internal Server Error, E5D1 - inner message Database error while creating dish., inner details are the SQL exception message.
- 500 Internal Server Error, E5G1 - malformed JSON syntax falls through to Unable to serve the REST request: CREATE_DISH.

- **Sample Request Body:**

```

1  {
2    "name": "Margherita",
3    "description": "Pizza con pomodoro, mozzarella e basilico",
4    "price": 8.5,
5    "ingredient_ids": [1, 2, 3],
6    "category": "Pizze"
7  }

```

Get All Dishes

Returns all dishes with nested ingredient objects.

- **URL:** /dish
 - **Method:** GET
 - **URL Parameters:** None
 - **Query Parameters:** None
 - **Headers:** Accept: application/json
 - **Cookies:** None
 - **Data Parameters:** None
 - **Success Response:** 200 OK
- Content:

```

1 {
2   "resource-list": [
3     {
4       "id": 1,
5       "name": "Bruschetta al pomodoro",
6       "description": "Pane tostato con pomodoro fresco e basilico",
7       "price": 5.5,
8       "ingredients": [
9         {
10          "id": 1,
11          "name": "Pomodoro San Marzano",
12          "allergens": [],
13          "is_frozen": false
14        },
15        {
16          "id": 3,
17          "name": "Basilico fresco",
18          "allergens": [],
19          "is_frozen": false
20        }
21      ],
22      "category": "Antipasti"
23    }
24  ]
25 }

```

- **Error Responses:**

- Common header errors: E4V1, E4V2.
- 500 Internal Server Error, E5D1 - inner message Database error while retrieving dishes., inner details are the SQL exception message.

Get Dish

Returns one dish by id.

- **URL:** /dish/{dish_id}
- **Method:** GET
- **URL Parameters:** dish_id - integer dish id
- **Query Parameters:** None
- **Headers:** Accept: application/json
- **Cookies:** None
- **Data Parameters:** None
- **Success Response:** 200 OK

Content:

```

1 {
2   "id": 2,
3   "name": "Spaghetti al pomodoro",
4   "description": "Spaghetti con salsa di pomodoro fresco e basilico",
5   "price": 9.0,
6   "ingredients": [
7     {
8       "id": 4,
9       "name": "Spaghetti di Gragnano",
10      "allergens": ["cereals", "eggs"],
11      "is_frozen": false
12    },
13    {
14      "id": 1,
15      "name": "Pomodoro San Marzano",

```

```

16     "allergens": [],
17     "is_frozen": false
18   }
19 ],
20 "category": "Primi"
21 }

```

- **Error Responses:**

- Common header errors: E4V1, E4V2.
- 400 Bad Request, E4V6 - inner message Invalid dish id format, inner details are the NumberFormatException message.
- 404 Not Found, E4R2 - inner message Dish with id 2 not found.
- 500 Internal Server Error, E5D1 - inner message Database error while retrieving dish., inner details are the SQL exception message.

Edit Dish

Updates a dish and replaces its ingredient bindings.

- **URL:** /dish
- **Method:** PUT
- **URL Parameters:** None
- **Query Parameters:** None
- **Headers:** Content-Type: application/json; Accept: application/json
- **Cookies:** auth_token HttpOnly cookie containing a valid JWT with role STAFF or ADMIN
- **Data Parameters:**

```

1 {
2   "id": 2,
3   "name": "Spaghetti al pomodoro fresco",
4   "description": "Spaghetti con salsa di pomodoro e basilico",
5   "price": 9.5,
6   "ingredient_ids": [4, 1, 3],
7   "category": "Primi"
8 }

```

Required fields are id, name, price, ingredient_ids, and category. name must be non-blank. Unlike create, this handler does not enforce price > 0; it only requires price != null. description is optional. Invalid category names, invalid ingredient ids, and duplicate ingredient ids surface as E5D1. Malformed JSON syntax is not explicitly handled and falls through to E5G1.

- **Success Response:** 200 OK

Content:

```

1 {
2   "id": 2,
3   "name": "Spaghetti al pomodoro fresco",
4   "description": "Spaghetti con salsa di pomodoro e basilico",
5   "price": 9.5,
6   "ingredients": [
7     {
8       "id": 4,
9       "name": "Spaghetti di Gragnano",
10      "allergens": ["cereals", "eggs"],
11      "is_frozen": false
12    },
13    {
14      "id": 1,
15      "name": "Pomodoro San Marzano",
16      "allergens": [],

```

```

17     "is_frozen": false
18   },
19   {
20     "id": 3,
21     "name": "Basilico fresco",
22     "allergens": [],
23     "is_frozen": false
24   }
25 ],
26 "category": "Primi"
27 }

```

- **Error Responses:**

- Common auth errors: 401 Unauthorized, E4A2; 403 Forbidden, E4A4.
- Common header errors: E4V1, E4V2, E4V3, E4V4.
- 400 Bad Request, E4V6 - inner message Missing required fields
- 400 Bad Request, E4V8 - inner message Malformed JSON in request body., inner details are Unexpected field: <field>.
- 404 Not Found, E4R2 - inner message Dish with id 2 not found.
- 500 Internal Server Error, E5D1 - inner message Database error while editing dish., inner details are the SQL exception message.
- 500 Internal Server Error, E5G1 - malformed JSON syntax falls through to Unable to serve the REST request: EDIT_DISH.

- **Sample Request Body:**

```

1 {
2   "id": 2,
3   "name": "Spaghetti al pomodoro fresco",
4   "description": "Spaghetti con salsa di pomodoro e basilico",
5   "price": 9.5,
6   "ingredient_ids": [4, 1, 3],
7   "category": "Primi"
8 }

```

Delete Dish

Deletes a dish by id.

- **URL:** /dish/{dish_id}
- **Method:** DELETE
- **URL Parameters:** dish_id - integer dish id
- **Query Parameters:** None
- **Headers:** Accept: application/json
- **Cookies:** auth_token HttpOnly cookie containing a valid JWT with role STAFF or ADMIN
- **Data Parameters:** None

The schema keeps order_dishes.dish_id as a foreign key without ON DELETE CASCADE, so deleting a dish referenced by an order fails with a database error (E5D1), not a conflict code.

- **Success Response:** 204 No Content

Content: None

- **Error Responses:**

- Common auth errors: 401 Unauthorized, E4A2; 403 Forbidden, E4A4.
- Common header errors: E4V1, E4V2.
- 400 Bad Request, E4V6 - inner message Invalid dish id format, inner details are the NumberFormatException message.
- 404 Not Found, E4R2 - inner message Dish with id 2 not found.

- 500 Internal Server Error, E5D1 - inner message Database error while deleting dish., inner details are the SQL exception message.

Get All Orders

Returns all orders. This route is available only to STAFF and ADMIN.

- **URL:** /order
 - **Method:** GET
 - **URL Parameters:** None
 - **Query Parameters:** None
 - **Headers:** Accept: application/json
 - **Cookies:** auth_token HttpOnly cookie containing a valid JWT with role STAFF or ADMIN
 - **Data Parameters:** None
- The order read DAOs use `ResultSet#getBoolean("is_liked")` without `wasNull()`, so database NULL values in `order_dishes.is_liked` are serialized as `false`.
- **Success Response:** 200 OK

Content:

```

1 {
2   "resource-list": [
3     {
4       "id": 2,
5       "status": "PENDING",
6       "total_price": 9.0,
7       "user_id": 4,
8       "promotion_id": null,
9       "dishes": [
10        {
11          "dish_id": 2,
12          "quantity": 1,
13          "is_liked": false
14        }
15      ]
16    }
17  ]
18 }
```

- **Error Responses:**
 - Common auth errors: 401 Unauthorized, E4A2; 403 Forbidden, E4A4.
 - Common header errors: E4V1, E4V2.
 - 500 Internal Server Error, E5D1 - inner message Database error while retrieving orders., inner details are the SQL exception message.

Create Order

Creates a new order.

- **URL:** /order
- **Method:** POST
- **URL Parameters:** None
- **Query Parameters:** None
- **Headers:** Content-Type: application/json; Accept: application/json
- **Cookies:** auth_token HttpOnly cookie containing a valid JWT
- **Data Parameters:**

```

1 {
2   "status": "PENDING",
```

```

3  "user_id": 4,
4  "promotion_id": 1,
5  "dishes": [
6    {
7      "dish_id": 1,
8      "quantity": 2,
9      "is_liked": null
10   }
11 ]
12 }

```

The parsed fields are `id`, `status`, `total_price`, `user_id`, `promotion_id`, and `dishes`, but the DAO ignores any submitted `total_price` and recomputes it from current dish prices. `status` must be one of the case-sensitive enum values `PENDING`, `COMPLETED`, or `CANCELLED`. `user_id` is required and must be a positive integer, but it is not cross-checked against the authenticated JWT user id; any authenticated caller can submit another user's id. `promotion_id` is optional and is an integer foreign key to `promotions.id`, not a promotion code. Each dish object may contain `dish_id`, `quantity`, and optional `is_liked`. Invalid user ids, invalid promotion ids, duplicate `(user_id, promotion_id)` combinations, duplicate dish ids within the same order, and other persistence failures surface as `E5D1`.

- **Success Response:** `201 Created`

Content:

```

1  {
2    "id": 6,
3    "status": "PENDING",
4    "total_price": 11.0,
5    "user_id": 4,
6    "promotion_id": 1,
7    "dishes": [
8      {
9        "dish_id": 1,
10       "quantity": 2,
11       "is_liked": null
12     }
13   ]
14 }

```

- **Error Responses:**

- Common auth error: `401 Unauthorized`, `E4A2`.
- Common header errors: `E4V1`, `E4V2`, `E4V3`, `E4V4`.
- `400 Bad Request`, `E4V6` - inner message `invalid status in order`. when `status` is missing or `null`.
- `400 Bad Request`, `E4V6` - inner message `Invalid user id in order` when `user_id` is missing, `null`, or `<= 0`.
- `400 Bad Request`, `E4V6` - inner message `Order must contain at least one item` when `dishes` is missing or empty.
- `400 Bad Request`, `E4V8` - inner message `Malformed JSON in request body.`, inner details are the parser message or `Unexpected key '<field>' in Order JSON` / `Unexpected key '<field>' in dish JSON`.
- `400 Bad Request`, `E4V8` - inner message `Invalid order.`, inner details are the `NotValidOrderException` message, for example `Invalid status: COMPLETE`.
- `400 Bad Request`, `E4V8` - inner message `Failed to create order. Please check the provided data.` when the DAO returns `null`, which happens when any referenced `dish_id` is not found during price lookup.
- `500 Internal Server Error`, `E5D1` - inner message `Unexpected database error: no. <sqlErrorCode>`, inner details are the SQL exception message. This includes invalid `user_id`, invalid `promotion_id`,

duplicate dish ids in one order, and reusing the same non-null promotion for the same user.

- **Sample Request Body:**

```
1 {
2   "status": "PENDING",
3   "promotion_id": 1,
4   "dishes": [
5     { "dish_id": 1, "quantity": 2, "is_liked": null }
6   ]
7 }
```

Get User Orders

Returns all orders placed by the authenticated user.

- **URL:** `/order/user`
- **Method:** `GET`
- **URL Parameters:** `None`
- **Query Parameters:** `None`
- **Headers:** `Accept: application/json`
- **Cookies:** `auth_token` `HttpOnly` cookie containing a valid JWT
- **Data Parameters:** `None`

The endpoint resolves the user from the dispatcher-set `user_id` request attribute. This route must be registered before `GET /order/{order_id}` in the router so that the literal segment `user` is matched first; if `/order/{order_id}` were registered earlier it would shadow this route.

- **Success Response:** `200 OK`

Content:

```
1 {
2   "resource-list": [
3     {
4       "id": 2,
5       "status": "PENDING",
6       "total_price": 9.0,
7       "user_id": 4,
8       "promotion_id": null,
9       "dishes": [
10        {
11          "dish_id": 2,
12          "quantity": 1,
13          "is_liked": false
14        }
15      ]
16    }
17  ]
18 }
```

- **Error Responses:**

- Common auth error: `401 Unauthorized`, `E4A2`.
- Common header errors: `E4V1`, `E4V2`.
- `404 Not Found`, `E4R2` - inner message `No orders found for user <id>` when the user has no orders.
- `400 Bad Request`, `E4V6` - inner message `Invalid user ID: must be a number.`, inner details are the `NumberFormatException` message, if the dispatcher-set `user_id` attribute is not numeric.
- `500 Internal Server Error`, `E5D1` - inner message `Unexpected database error: no. <sqlErrorCode>`, inner details are the SQL exception message.

Get Order

Returns one order by id.

- **URL:** `/order/{order_id}`
- **Method:** `GET`
- **URL Parameters:** `order_id` - integer order id
- **Query Parameters:** `None`
- **Headers:** `Accept: application/json`
- **Cookies:** `auth_token` `HttpOnly` cookie containing a valid JWT
- **Data Parameters:** `None`

Any authenticated user can retrieve any order by id; the code performs no ownership check against the JWT `user_id`.

- **Success Response:** `200 OK`

Content:

```
1 {
2   "id": 2,
3   "status": "PENDING",
4   "total_price": 9.0,
5   "user_id": 4,
6   "promotion_id": null,
7   "dishes": [
8     {
9       "dish_id": 2,
10      "quantity": 1,
11      "is_liked": false
12    }
13  ]
14 }
```

- **Error Responses:**

- Common auth error: `401 Unauthorized`, `E4A2`.
- Common header errors: `E4V1`, `E4V2`.
- `400 Bad Request`, `E4V6` - inner message `Invalid order ID <value>`, inner details are the `NumberFormatException` message.
- `404 Not Found`, `E4R2` - inner message `Order with id 2 not found`.
- `500 Internal Server Error`, `E5D1` - inner message `Unexpected database error: no. <sqlErrorCode>`, inner details are the SQL exception message.

Update Order Status

Updates the status of an order.

- **URL:** `/order/{order_id}/{order_status}`
- **Method:** `PUT`
- **URL Parameters:** `order_id` - integer order id; `order_status` - one of `PENDING`, `COMPLETED`, `CANCELLED`
- **Query Parameters:** `None`
- **Headers:** `Content-Type: application/json`; `Accept: application/json`
- **Cookies:** `auth_token` `HttpOnly` cookie containing a valid JWT with role `STAFF` or `ADMIN`
- **Data Parameters:** `None`

Although this endpoint has no request body, `Content-Type: application/json` is still required because the shared base class treats every `PUT` as body-bearing. `order_status` is parsed with `OrderStatus.valueOf`, so matching is case-sensitive. If the target order does not exist, the current DAO throws `SQLException("Failed to update order status")`; the handler therefore returns `500 / E5D1`, not `404`.

- **Success Response:** 200 OK

Content:

```

1 {
2   "id": 2,
3   "status": "COMPLETED",
4   "total_price": 9.0,
5   "user_id": 4,
6   "promotion_id": null,
7   "dishes": [
8     {
9       "dish_id": 2,
10      "quantity": 1,
11      "is_liked": false
12    }
13  ]
14 }
```

- **Error Responses:**

- Common auth errors: 401 Unauthorized, E4A2; 403 Forbidden, E4A4.
- Common header errors: E4V1, E4V2, E4V3, E4V4.
- 400 Bad Request, E4V6 - inner message Invalid order ID: must be a number, inner details are the NumberFormatException message.
- 400 Bad Request, E4V6 - inner message Invalid order status, inner details are the IllegalArgumentException message from OrderStatus.valueOf.
- 500 Internal Server Error, E5D1 - inner message Unexpected database error: no. <sqlErrorCode>, inner details are the SQL exception message. This is also the current response for nonexistent order ids.

Delete Order

Deletes an order.

- **URL:** /order/{order_id}/
- **Method:** DELETE
- **URL Parameters:** order_id - integer order id
- **Query Parameters:** None
- **Headers:** Accept: application/json
- **Cookies:** auth_token HttpOnly cookie containing a valid JWT with role ADMIN
- **Data Parameters:** None

The trailing slash is part of the configured route. /rest/order/4/ matches; /rest/order/4 does not and instead falls through to the dispatcher-level 404/E4R1 unknown-resource response.

- **Success Response:** 204 No Content

Content: None

- **Error Responses:**

- Common auth errors: 401 Unauthorized, E4A2; 403 Forbidden, E4A4.
- Common header errors: E4V1, E4V2.
- 400 Bad Request, E4V6 - inner message Invalid format of order ID <value> to be deleted, inner details are the NumberFormatException message.
- 404 Not Found, E4R2 - inner message Order with id 4 not found for deletion
- 500 Internal Server Error, E5D1 - inner message Unexpected database error: no. <sqlErrorCode>, inner details are the SQL exception message.

Create Promotion

Creates a promotion.

- **URL:** /promotion
- **Method:** POST
- **URL Parameters:** None
- **Query Parameters:** None
- **Headers:** Content-Type: application/json; Accept: application/json
- **Cookies:** auth_token HttpOnly cookie containing a valid JWT with role ADMIN
- **Data Parameters:**

```

1 {
2   "code": "BENVENUTO15",
3   "discount": 15.0,
4   "description": "Sconto 15% per i nuovi clienti",
5   "valid_from": "2026-05-01T00:00:00",
6   "valid_to": "2026-12-31T23:59:59"
7 }

```

Required fields are `code`, `discount`, `description`, `valid_from`, and `valid_to`. `discount` must satisfy $0 < \text{discount} \leq 100$. `valid_to` must not be before `valid_from`; equality is not rejected by the handler, but it violates the database CHECK (`valid_to > valid_from`) constraint and therefore becomes E5D1. Date-time parsing uses `LocalDateTime.parse`, so the expected format is ISO-8601 local date-time such as `2026-05-01T00:00:00`. Duplicate promotion codes are not explicitly mapped to 409; they surface as 500 / E5D1.

- **Success Response:** 201 Created
Content:

```

1 {
2   "code": "BENVENUTO15",
3   "discount": 15.0,
4   "description": "Sconto 15% per i nuovi clienti",
5   "valid_from": "2026-05-01T00:00:00",
6   "valid_to": "2026-12-31T23:59:59"
7 }

```

- **Error Responses:**

- Common auth errors: 401 Unauthorized, E4A2; 403 Forbidden, E4A4.
- Common header errors: E4V1, E4V2, E4V3, E4V4.
- 400 Bad Request, E4V6 - inner message Missing required fields
- 400 Bad Request, E4V6 - inner message Invalid discount: it must be between 0 and 100.
- 400 Bad Request, E4V6 - inner message Invalid promotion dates: validTo must be after validFrom
- 400 Bad Request, E4V8 - inner message Malformed JSON in request body., inner details are Unexpected field: <field>.
- 500 Internal Server Error, E5D1 - inner message Failed to create promotion. if the DAO returns no row.
- 500 Internal Server Error, E5D1 - inner message Unexpected database error: no. <sqlErrorCode>, inner details are the SQL exception message. This includes duplicate codes and equality failures on valid_from/valid_to.
- 500 Internal Server Error, E5G1 - malformed JSON syntax or invalid LocalDateTime strings fall through to Unable to serve the REST request: NEW_PROMOTION.

- **Sample Request Body:**

```

1 {
2   "code": "BENVENUTO15",
3   "discount": 15.0,
4   "description": "Sconto 15% per i nuovi clienti",
5   "valid_from": "2026-05-01T00:00:00",
6   "valid_to": "2026-12-31T23:59:59"
7 }

```

Get All Promotions

Returns all promotions. This route is available only to `STAFF` and `ADMIN`.

- **URL:** `/promotion`
- **Method:** `GET`
- **URL Parameters:** `None`
- **Query Parameters:** `None`
- **Headers:** `Accept: application/json`
- **Cookies:** `auth_token` HttpOnly cookie containing a valid JWT with role `STAFF` or `ADMIN`
- **Data Parameters:** `None`
The DAO returns all rows from `promotions` without filtering by current date or validity window.
- **Success Response:** `200 OK`
Content:

```
1 {
2   "resource-list": [
3     {
4       "code": "BENVENUTO10",
5       "discount": 10.0,
6       "description": "Sconto 10% per i nuovi clienti",
7       "valid_from": "2026-01-01T00:00:00",
8       "valid_to": "2026-12-31T23:59:59"
9     }
10  ]
11 }
```

- **Error Responses:**
 - Common auth errors: `401 Unauthorized`, `E4A2`; `403 Forbidden`, `E4A4`.
 - Common header errors: `E4V1`, `E4V2`.
 - `404 Not Found`, `E4R2` - inner message `No promotions found.` when the DAO returns `null`.
 - `500 Internal Server Error`, `E5D1` - inner message `Unexpected database error: no. <sqlErrorCode>`, inner details are the SQL exception message.

Get Promotion By Code

Returns a promotion by code.

- **URL:** `/promotion/{promotion_code}`
- **Method:** `GET`
- **URL Parameters:** `promotion_code` - promotion code string
- **Query Parameters:** `None`
- **Headers:** `Accept: application/json`
- **Cookies:** `None`
- **Data Parameters:** `None`
This endpoint is public and does not filter promotions by validity dates; it simply looks up the row by `code`.
- **Success Response:** `200 OK`
Content:

```
1 {
2   "code": "BENVENUTO10",
3   "discount": 10.0,
4   "description": "Sconto 10% per i nuovi clienti",
5   "valid_from": "2026-01-01T00:00:00",
6   "valid_to": "2026-12-31T23:59:59"
7 }
```

- **Error Responses:**

- Common header errors: `E4V1`, `E4V2`.
- `404 Not Found`, `E4R2` - inner message `Promotion with code BENVENUTO10 not found.`
- `500 Internal Server Error`, `E5D1` - inner message `Unexpected database error: no. <sqlErrorCode>`, inner details are the SQL exception message.

Check Promotion Usage

Returns whether the authenticated user has already used a promotion code on an order.

- **URL:** `/promotion/{promotion_code}/usage`
- **Method:** `GET`
- **URL Parameters:** `promotion_code` - promotion code string
- **Query Parameters:** `None`
- **Headers:** `Accept: application/json`
- **Cookies:** `auth_token` `HttpOnly` cookie containing a valid JWT
- **Data Parameters:** `None`

The endpoint uses the JWT `user_id`; there is no user id path or query parameter in the configured route. The DAO performs `COUNT(*)` over `orders JOIN promotions`, so nonexistent promotion codes also return `false` rather than `404`.

- **Success Response:** `200 OK`
Content:

```
1 {
2   "is_already_used": true
3 }
```

- **Error Responses:**
 - Common auth error: `401 Unauthorized`, `E4A2`.
 - Common header errors: `E4V1`, `E4V2`.
 - `500 Internal Server Error`, `E5D1` - inner message `Unexpected database error`, inner details are the SQL exception message.

6 Group Members Contribution

Baldan Fabio (2203580) has contributed to this project by:

1. Designing and implementing the resource model layer: `AbstractResource`, `User`, `Dish`, `Order`, `OrderDish`, `Category`, `Promotion`, `Message`, and `ResourceList`, including full JSON serialisation and deserialisation logic;
2. Implementing `AbstractRR`, the shared base class for all REST resources, handling `Accept` and `Content-Type` validation, error envelope formatting, and the dispatch lifecycle;
3. Implementing `EditUserRR` and the corresponding `EditUserDAO`;
4. Writing a Docker Compose file with Tomcat and all the necessary services, implementing Docker secrets for secure credential management;
5. Designing and maintaining the database schema (`CREATE.sql`) through multiple revisions;
6. Implementing the `AuthFlowFilter` servlet filter to redirect already-authenticated users.

Garberino Alvisè (2196387) has contributed to this project by:

1. Designing and maintaining the ER diagram (`ER.drawio`) and deriving the relational database schema from it, including foreign-key constraints and `ON DELETE` rules;

2. Implementing all five Dish DAOs: `CreateDishDAO`, `DeleteDishDAO`, `EditDishDAO`, `GetDishDAO`, and `GetAllDishesDAO`;
3. Implementing the Ingredient read DAOs: `GetIngredientDAO` and `GetAllIngredientsDAO`;
4. Implementing the `RestResource` interface and the initial `RestDispatcherServlet` with basic HTTP route dispatching;
5. Implementing `GetUserRR` and contributing to authentication cookie validation in the web filter.

Merja Klaudio (2197815) has contributed to this project by:

1. Implementing `JWTUtil` for JWT generation, signing, and validation, and integrating it into the authentication flow;
2. Implementing `AbstractDAO` and `ConnectionPoolSingleton`, providing the shared data-access foundation for all DAOs;
3. Implementing the complete REST resource layer for Users (`RegisterUserRR`, `AuthenticateUserRR`, `LogoutUserRR`, `DeleteUserRR`), Categories (`CreateCategoryRR`, `DeleteCategoryRR`, `GetAllCategoriesRR`), Ingredients (`CreateIngredientRR`, `EditIngredientRR`, `DeleteIngredientRR`), and Dishes (`CreateDishRR`, `EditDishRR`, `GetDishRR`, `GetAllDishesRR`);
4. Implementing the corresponding Category DAOs: `CreateCategoryDAO`, `DeleteCategoryDAO`, and `GetAllCategoriesDAO`;
5. Structuring the project layout, managing Maven dependencies and plugins, and enforcing code style via the Spotless plugin.

Padoan Giancarlo (2188345) has contributed to this project by:

1. Designing and implementing `ErrorCodes` and `Actions`, providing a unified taxonomy of REST error codes and action identifiers used throughout the application;
2. Implementing `Validator` for server-side regex validation of email addresses and passwords;
3. Implementing `RegisterUserDAO` and contributing to the `GetOrderDAO`;
4. Implementing the complete REST resource layer for Orders (`CreateOrderRR`, `DeleteOrderRR`, `EditOrderStatusRR`, `GetOrderRR`, `GetAllOrdersRR`, `GetUserOrdersRR`) and Promotions (`NewPromotionRR`, `GetPromotionRR`, `CheckPromotionRR`, `CheckUsageRR`);
5. Implementing all Promotion DAOs: `NewPromotionDAO`, `GetPromotionDAO`, `CheckPromotionDAO`, and `CheckPromotionUsageDAO`;
6. Implementing `DashboardFilter` and custom exception classes (`NotValidOrderException`, `MalformedJSONException`).

Sanavia Thomas (2197484) has contributed to this project by:

1. Implementing the `DataAccessObject` base interface and `LogContext` for structured logging across the application;
2. Implementing all six Order DAOs: `CreateOrderDAO`, `DeleteOrderDAO`, `EditOrderStatusDAO`, `GetOrderDAO`, `GetAllOrdersDAO`, and `GetUserOrdersDAO`, including transaction management for multi-step order creation;
3. Implementing the centralised `Router` class for route registration and dispatch;
4. Implementing `DashboardServlet`, `IngredientsServlet`, `ResourceList`, and the `UnexpectedKeyException` JSON exception;
5. Bug-fixing and adjusting misconfigurations of the Docker Compose file, and contributing to `AbstractRR` refactoring (Content-Type handling and transaction flow);
6. Writing the REST API documentation, covering the summary table and the detailed endpoint specifications for Orders and Promotions.

7 How to run the application

The application is designed to run in a Docker environment. All the material needed to build and run is included in the repository. Read the `README.md` file carefully for detailed instructions on how to build the WAR and start the containers.

The `docker-compose.yml` file orchestrates three services:

- **web**: the Tomcat application server, built from the local `Dockerfile` and exposed on port `8080`. It mounts the compiled `target/taptable.war`, the application logs directory and the Tomcat configuration files (`server.xml`, `tomcat-users.xml`, `context.xml`). It depends on the `db` service being healthy before starting.
- **db**: a PostgreSQL container, exposed on port `5432`. The database schema is initialised automatically from `db/CREATE.sql` on first startup. Database credentials are provided via environment variables. Docker secrets support is also configured and it is stored in `secrets/postgres_password.txt`.
- **pgadmin**: a pgAdmin instance exposed on port `5050`, pre-configured with server-mode disabled and setted with default email and password (even if it is not required the authentication). It is useful for inspecting the database during development and debugging phases.

Before running in any non-development environment, assure that the `jwt.secret` context parameter in `src/main/webapp/WEB-INF/web.xml` have been replaced with a strong random secret.